

Mira — Workflow Architecture & Node Reference

Overview

Mira is a multi-agent, RAG-grounded project-intelligence assistant built in **n8n**, covering the three core capabilities required by the capstone brief (project planning, risk assessment, status reporting) plus one extended capability (daily milestone alerting) and human-in-the-loop governance on write actions. The system is split into two separate n8n workflows:

1. **Ingestion Pipeline** (MiraProject-PineconeIngestion.json) — pulls project files from a Google Drive folder, chunks and embeds them, and upserts them into a Pinecone vector index. Runs on its own schedule, independent of user requests.
2. **Retrieval Pipeline** (MiraProject-AIPoweredIntelligentAssistant-NewRefined.json) — the live, request-serving workflow. A single **Classifier (Orchestrator) Agent** routes each user message to one or more of four **specialist agents** (Planner, Risk Assessor, Status Reporter, Stakeholder Update), each of which pulls grounding context from Pinecone as an LLM *tool call* (retrieve-as-tool, not a fixed top-k pre-fetch), and each of which is followed by deterministic (non-LLM) validation/parsing code before a response is returned. A fifth, schedule-triggered agent (Milestone Tracker) runs daily independent of user input.

Orchestration pattern

The brief's three reference patterns were Sequential Pipeline, Router/Dispatcher, and Hierarchical. **Mira implements the Router/Dispatcher pattern**: the Classifier Agent (N6) inspects the user's message and emits a routes array (it supports **multi-intent** messages — a single message can be routed to more than one specialist), which a Code node (N7) explodes into one workflow item per route before a native n8n Switch node (N9) dispatches each item to its specialist agent. This matches the brief's own recommendation ("A Router pattern fits well because user requests vary") while extending it to handle compound requests.

Architecture summary

Layer	Nodes	Purpose
Entry	N1 (chat trigger) / N1: Webhook, N2: User Input	Normalizes chat-widget and raw webhook input into one shape; tags source (chat vs webhook) so the final response routes back correctly
Orchestration	N6: Classifier Agent, N6: Simple Memory, N6a, N7, N8, N9	Classifies intent (plan / risk / status / unclear), validates the JSON, explodes multi-intent, routes
Specialist agents	N11 Planner, N16 Risk Assessor, N25 Status Reporter, M2 Milestone Tracker	Each is an @n8n agent node grounded via a dedicated Pinecone retrieve-as-tool node, backed by gpt-4o-mini
Deterministic guardrails	N17d, N26b, N26d, plus every parser Code node	Non-LLM JavaScript that re-verifies, dedupes, re-buckets, or caps the LLM's own JSON output against the live Google Sheets data before it reaches the user — added because the LLM's own gatekeeping/bucketing proved unreliable in testing

Mira — Workflow Architecture & Node Reference

Layer	Nodes	Purpose
Human-in-the-loop	N21b: New Risk Approval	Slack native sendAndWait double-approval (Approve/Deny) gate before any new risk is written to the register
Persistence	N17b/N22 (Risk Register), N26c (Task Board) — Google Sheets; Pinecone (mira-pineconeindex) — vector store	Live system-of-record reads/writes, separate from the RAG index
Output	Final Gate, Respond to Webhook, Post Message to Slack	Every response is posted to Slack channel mira-updates; webhook-sourced requests additionally get a synchronous HTTP response
Observability	LangSmith tracing (env vars) + evaluators	Full request/response tracing plus 4 offline evaluators (Hallucination, Assertions, Answer Relevance, Bias & Fairness)
Extended capability	Fine-tuning of the Risk Assessor's base model on corrected bug/fix examples	Optional enhancement per brief; see Fine-Tuning

Note: The sections below documents every node, in execution order, with parameters and full prompt text as configured in the respective JSON workflows.

Pipeline 1 — Pinecone Ingestion Workflow

Workflow file: `MiraProject-PineconeIngestion.json`: Pinecone ingestion - Executed as a separate n8n workflow

Node	Type	Parameters
R1: Trigger Pinecone Embedding	scheduleTrigger (v1.4)	Interval: every 1 minute
R2: Check Index Stats (HTTP Request)	httpRequest (v4.5)	POST <code>https://mira-pineconeindex-unyv3gm.svc.aped-4627-b74a.pinecone.io/describe_index_stats</code> ; auth: generic Header Auth (Header Auth account); body {}
R3: IF Namespace Exists?	if (v2.3)	Condition: <code>{{ Object.keys(\$json.namespaces {}).includes('NS_miraprojectplanningfiles') }} equals true (boolean, strict)</code> . True → R4 (wipe then re-ingest). False → R5 (first-time ingest)
R4: HTTP Request	httpRequest (v4.5)	POST <code>.../vectors/delete</code> ; body <code>{ "deleteAll": true, "namespace": "NS_miraprojectplanningfiles" }</code> — clears the namespace before a full re-ingest, avoiding stale/duplicate vectors
R5: Search Drive Folder	googleDrive (v3)	resource: fileFolder; folderId: <code>1y-kny5YLFQzn453ldyhul5fboUDPzMbv ("CapstoneProject-Mira")</code> — lists all files in the project's Drive folder
R6: Download New File(s)	googleDrive (v3)	operation: download; fileId: <code>{{ \$json.id }}</code> ; Google-native docs are auto-converted to PDF on download (<code>googleFileConversion.docsToFormat: application/pdf</code>)
R7: Loop Over Items	splitInBatches (v3)	Iterates one file at a time into R8; empty-batch output left unwired

Mira — Workflow Architecture & Node Reference

Node	Type	Parameters
R8: Pinecone: Insert Documents	vectorStorePinecone (v1.3)	mode: insert; index mira-pineconeindex; namespace: NS_miraprojectplanningfiles; loops back into R7 after each insert
R8: Embeddings OpenAI (feeds R8 insert, ai_embedding)	embeddingsOpenAi (v1.2)	dimensions: 768
R8: Default Data Loader (feeds R8 insert, ai_document)	documentDefaultDataLoader (v1.1)	dataType: binary; textSplittingMode: custom; tags each chunk's source metadata by filename pattern: includes('Risk Register') → 'risk_register', includes('Task Board') → 'task_board', includes('Timeline') → 'timeline', else 'drive_doc' — this is the metadata field every retrieval-side tool later filters on
R8: Character Text Splitter (feeds Data Loader, ai_textSplitter)	textSplitterCharacterTextSplitter (v1)	separator: \n; chunkOverlap: 250

Flow: R1 → R2 → R3 → (R4 →) R5 → R6 → R7 ⇌ R8 (insert, loop).

Design notes

- The namespace-exists check (R2/R3) makes re-runs: if the namespace already has vectors, it is wiped (R4) before the fresh Drive files are re-embedded, so the index never accumulates duplicate or stale chunks across re-ingestion runs.
- All ingestion nodes share credentials: **Google Drive account** (Drive), **OpenAI account** (embeddings), **Pinecone account** (vector store), **Header Auth account** (raw Pinecone REST calls for stats/delete).

Pipeline 2 — Retrieval Workflow: Entry, Routing & Validation

Workflow file: MiraProject-AIPoweredIntelligentAssistant-NewRefined.json

Entry nodes

Node	Type	Parameters
N1: When chat message received	chatTrigger (v1.4)	responseMode: lastNode — the n8n built-in chat widget entry point
N1: Webhook	webhook (v2.1)	httpMethod: POST; responseMode: responseNode — a second, raw-HTTP entry point for programmatic callers
N2: User Input	set (v3.5)	Normalizes both entry shapes into one: chatInput = {{ \$json.chatInput \$json.body.input }}; sessionId = {{ \$json.sessionId \$json.body.sessionId 'langsmith-' + \$execution.id }}; source = {{ \$json.chatInput ? 'chat' : 'webhook' }} — the source tag is read again at the very end of the workflow (Final Gate) to decide whether a synchronous HTTP response is also required

Mira — Workflow Architecture & Node Reference

N1 / N2: Entry Triggers & the ask-mira.html Test Client

Two independent triggers converge into the same downstream pipeline, normalized by a single Set node before anything else runs:

1. N1: When chat message received (@n8n/n8n-nodes-langchain.chatTrigger, v1.4) — the built-in n8n chat widget, used for manual/test runs.
2. N1: Webhook1 (n8n-nodes-base.webhook, v2.1) — httpMethod: POST, path: Mirabot-NewRefined, responseMode: responseNode. This is the primary entry point for any external caller, including the attached ask-mira.html test client.
3. N2: User Input (n8n-nodes-base.set, v3.5) normalizes both trigger shapes into one: chatInput = {{ \$json.chatInput || \$json.body.input }}, sessionId = {{ \$json.sessionId || \$json.body.sessionId || 'langsmith-' + \$execution.id }}, and source = {{ \$json.body !== undefined ? 'webhook' : 'chat' }}. Every downstream agent prompt reads N2's chatInput field, and Final Gate later reads N2's own source field to decide how to deliver the response.

ask-mira.html (attached) is a small standalone static page built as a lightweight, dependency-free test/demo client for the webhook entry point — independent of the n8n chat widget, useful for verifying the deployed workflow without opening the n8n editor. It POSTs a JSON body { "input": <question> } to a configurable Webhook URL field, defaulting to `http://localhost:5678/webhook/Mirabot-NewRefined` — matching N1: Webhook1's path— and displays the raw response text. Its inline script comments record the same chatInput vs body.input distinction N2 implements, and note that Respond to Webhook is configured with Respond With: Text (not JSON), so the page reads the response body as plain text.

N6: Classifier Agent (the Orchestrator)

- **Type:** @n8n agent node (v3.1), executeOnce: true
- **Prompt (user text):** {{ \$('N2: User Input').item.json.chatInput }}
- **LLM:** OpenAI Chat Model → gpt-4o-mini (default temperature)
- **Memory:** N6: Simple Memory (memoryBufferWindow) wired in
- **System message (verbatim):**

You are the Classifier Agent for Mira, a project intelligence assistant. Classify the user's request into exactly one category based on intent, not keywords alone:

- "plan": the user wants a project plan generated
- "risk": the user wants a risk assessment or wants to know existing risks
- "status": the user wants a status report, task summary, or milestone check
- "stakeholder_update": the user wants a written update/email/summary communication for stakeholders — distinct from "status," which wants structured task/milestone data, not prose. Trigger words: "stakeholder update," "progress email," "write an update," "summary for leadership/stakeholders."
- "unclear": the request does not clearly map to any of the above, or lacks enough detail to route

If the request appears to be answering a previous clarifying question (for example, it only supplies scope, timeline, team size, or similar specifics with no other context), use the conversation history to classify it the same way as the original request it's completing, not as a fresh unclear message.

Mira — Workflow Architecture & Node Reference

DISAMBIGUATION: A request about a person's departure, leaving, or stepping back – even if it mentions "sprints" or task-related words – is a RISK question about organizational impact, not a STATUS question about task progress. Status is about the current state of existing tasks; Risk covers what happens if a person or resource is lost. When in doubt between the two for a personnel-change scenario, choose "risk."

A request for "risks" or "top N risks" naming only "this project" or "the project" (no explicit project name) is still clearly a RISK request – route it to "risk" regardless of whether a specific project name was given. Whether enough detail exists to actually answer it is the Risk Assessor's own job to determine afterward, not something you need to pre-filter here.

A request asking for a report, summary, or update phrased as something to SEND or SHARE with others (an email, a written update, a message "for the team" or "for stakeholders") is "stakeholder_update," even if it also references sprint or task-related content – the deciding factor is whether the user wants structured data back (status) or a written communication they can send as-is (stakeholder_update).

This disambiguation guidance is about WHICH category a request belongs to, not whether it has enough DETAIL to be answered. A request that is clearly plan-shaped, risk-shaped, status-shaped, or stakeholder-update-shaped – even if vague or underspecified – should still route to that category. Only use "unclear" when the request's INTENT itself cannot be determined (i.e., you cannot tell which of the four categories it belongs to at all) – never use "unclear" as a substitute for "insufficiently detailed." Each specialist agent has its own logic for flagging vague requests within its category; that is not your job.

Output strict JSON: { "routes": ["plan"|"risk"|"status"|"stakeholder_update", ...], "reason": string }

- Output schema uses a routes array, not a single string — supporting multi-intent messages (e.g. "give me a status report and assess the risks" routes to both status and risk). The routes array covers four categories (plan, risk, status, stakeholder_update) plus unclear; see the Stakeholder Update Agent Sub-Pipeline section below for the stakeholder_update branch.

Parsing, multi-intent explosion, validation, routing

Node	Type	Behavior
N6a: JS-Parse & Validate JSON	code	Strips ```json fences, JSON.parses the Classifier's output, re-attaches chatInput/sessionId/source, sets _valid/_parseError
N7: Multi-intent message	code	return \$json.routes.map(r => ({ json: { ...\$json, route: r } })) — turns one classification into one workflow item per detected route, so everything downstream runs once per intent
N8: IF: JSON Valid?	if	Condition: {{ \$json._valid }} is not true → routes to E: Notify Parse Error to Slack (posts the parse error to the #errors Slack channel). Otherwise → N9
N9: Switch: Route by Intent	switch	mode: expression; {{ ['plan','risk','status','unclear'].includes(\$json.route) ? [...].indexOf(\$json.route) : 3 }} — 4 outputs: 0=plan→N11, 1=risk→N16 and N17b (parallel), 2=status→N25 and N26c (parallel), 3=unclear→N10
N10: Ask for Clarification	set	output = "🔍 I'm not sure what you're looking for yet. Here's how I read the request: {{ \$json.reason }}. I can put together a project plan, a risk assessment, or a status report for the ABCDE Ltd. AI Adoption Project: which one would you like?" → Final Gate

Mira — Workflow Architecture & Node Reference

Planner Agent Sub-Pipeline

N11: Planner Agent

- **Type:** @n8n agent node (v3.1)
- **LLM:** OpenAI Chat Model2 → gpt-4o-mini, temperature: 0
- **Tool:** R9: Pinecone Get (Planner Context) — retrieve-as-tool mode, index mira-pineconeindex, topK: 12, namespace NS_miraprojectplanningfiles, metadata filter source \$in ["drive_doc", "timeline", "task_board"]; tool description: *"Retrieve Documents from Pinecone to set context for Planning Agent."* Embeddings: R9: Embeddings OpenAI (OpenAI, 768-dim)
- **Prompt (user text, verbatim):**

User request: {{ \$('N2: User Input').first().json.chatInput }}

CRITICAL STEP FOR PLANNING EXECUTION:

1. Cross-examine the "User request" against the actual project descriptions, roadmaps, and scope documents provided in your retrieved context.
2. Ground every phase, milestone, and activity strictly in the facts present within those files. Do not invent milestones or dependencies that are completely absent from the source context.
3. If specific hard deadlines or numeric NFRs (Non-Functional Requirements) are defined in the text, they must be preserved exactly as targets in the output timeline estimates.

- **System message (verbatim, complete):**

You are the Planner Agent for Mira, operating within a logistics/AI adoption project. Your objective is to translate user planning requests into a structured, high-level project roadmap utilizing the project description documents, scopes, and timelines retrieved via your vector context window.

STEP 0: CONTEXT SUMMARY FIRST

Before generating the phase breakdown, briefly state what you're planning and what it's grounded in — name the specific project, scope, or component being planned, and reference the actual retrieved source (e.g., "the project description," "the current timeline") that informs it. If the user's request includes a specific question or constraint alongside the plan ask (e.g., "plan this out, but can we hit the Q1 deadline," "does this account for the security review dependency"), address that specific point directly here too, before the phases below. Populate this in the "answer" field. Keep it to 2-4 sentences — this is a grounding summary, not a restatement of the full roadmap.

CORE PRINCIPLES (STRICT COMPLIANCE REQUIRED)

- * NO HALLUCINATED FACTS: Ground all phases, activities, and milestones strictly in the retrieved text. If a planning timeline or specific toolset is missing from the database, do not guess it. Note it strictly as an assumption or an open requirement placeholder in the activities list.
- * HARD NUMBERS SURVIVE: Any numeric constraints, deadlines, budget thresholds, or delivery targets found in the project context must be explicitly included in the timeline estimates or phase descriptions without being softened. A numeric constraint supplied by the user (e.g., a duration) still requires actual scope and goals to plan around — a number alone is never a substitute for those (see GATEKEEPER ROUTING LAW).
- * TRACEABILITY BY DEFAULT: Every generated phase must tie back to actual initiatives or workstreams outlined in the retrieved documents.

STEP-BY-STEP PLANNING PIPELINE:

1. **CONTEXT EXTRACTION & GROUNDING:** Evaluate the user's input request against all retrieved context data. Identify the core system, module, or stream the user intends to plan out. Verify this identification LITERALLY against the retrieved text — the specific project, scope, or component named in the request must actually appear in the retrieved documents, not merely feel thematically related to what was retrieved.
2. **ROADMAP GENERATION:** Map the identified requirements into logical, structured project phases. Keep the elements focused on macro-level milestones and key deliverables defined by the project constraints.

Mira — Workflow Architecture & Node Reference

MANDATORY KPI/METRIC CHECK (perform this before finalizing your output): Before generating "phases," scan the retrieved project description specifically for any numeric success metrics, targets, percentages, or KPIs (e.g., "reduce delivery delays by 15%," "lower costs by 10%") — these are distinct from task-level deadlines and are easy to overlook since they describe the project's overall goal rather than a specific phase. If any such metric exists in the retrieved text, it **MUST** appear explicitly in at least one phase's activities, deliverables, or timeline_estimate (e.g., as a measurable success criterion for a pilot or evaluation phase) — do not generate a plan that omits a real, retrieved numeric target. This is a specific instance of **HARD NUMBERS SURVIVE**, verified explicitly rather than left to general compliance.

3. GATEKEEPER ROUTING LAW (sufficient_detail):

- A request is sufficiently detailed **ONLY** if it specifies, or can be directly tied to, a specific scope, deliverable, or initiative that is actually documented in the retrieved project files — not merely because retrieved documents happen to exist. A generic, one-line request ("build a chatbot," "make an app") does **NOT** become sufficient just because the retrieved project description contains other, unrelated scopes, team sizes, or timelines. The connection must be to the **SAME** thing the user is asking about, not just to any content in the index.
- A timeline or duration alone (e.g., "a 2-week project") is **NEVER** sufficient by itself. A concrete number is not a substitute for scope, goals, or deliverables — set sufficient_detail to false if only a duration is given with nothing else to plan around.
- Set "sufficient_detail" to false if the request lacks a specific, identifiable scope, goal, or deliverable, whether because the request itself is generic or because it supplies only a timeline/duration. List the exact items actually missing under "missing_info" (typically scope, goals, team size, or timeline — whichever the request genuinely lacks).
- Set "sufficient_detail" to true only if the request names or clearly identifies a specific project, component, or initiative that is directly documented in the retrieved files (e.g., "the AI Adoption Project," a specific named phase or module), or if the user supplies explicit structural detail themselves (scope, timeline, and team size, or equivalent).

GATEKEEPER FAILURE MODE TO AVOID: A common mistake is treating the broader retrieved project's own documented phases/activities as if they were the **SPECIFIC** scope the user's one-line request named — even when those phases sound thematically related. "Build a chatbot" is not made sufficient by a retrieved AI Adoption Project that happens to include phases like "evaluate AI use cases" or "shortlist AI use cases" — a chatbot is **A KIND OF** AI use case, but the request itself never specified which use case, what it should do, who it serves, or on what timeline. Before setting sufficient_detail to true, ask: does the retrieved text document **THIS EXACT** initiative (e.g., "chatbot project," with its own named scope, deliverables, and dates) — or am I borrowing a different, broader initiative's plan and relabeling it? If it's the latter, sufficient_detail **MUST** be false.

Output an unquoted, strict JSON object matching this schema exactly:

```
{
  "sufficient_detail": boolean,
  "answer": "string — a 2-4 sentence grounding summary: what's being planned, what retrieved source it's based on, and a direct response to any specific question or constraint the user asked alongside the plan request.",
  "missing_info": ["string"],
  "phases": [
    {
      "phase_name": "string",
      "activities": ["string"],
      "milestones": ["string"],
      "deliverables": ["string"],
      "timeline_estimate": "string"
    }
  ]
}
```

Downstream chain

Node	Type	Behavior
N12: Parse Plan JSON	code	Strips fences, JSON.parses N11's output, re-attaches session fields, sets _valid
N13: IF: Plan Sufficient Detail?	if	{{ \$json.sufficient_detail }} is truthy → N15; else → N14
N14: Ask for More Plan Detail	set	output = "🔍 I need a bit more detail to generate a plan. \n\nHere's what's missing: {{ \$json.missing_info.join(", ") }}. \n\nCould you share the project's scope and

Mira — Workflow Architecture & Node Reference

Node	Type	Behavior
		timeline?" → Final Gate
N15: Return Plan	set	Formats answer + each phase (phase_name, timeline_estimate, activities, milestones, deliverables) into a single Markdown-ish output string, prefixed  → Final Gate

Risk Assessor Agent Sub-Pipeline (N16) — fine-tuning target

N16: Risk Assessor Agent

- **Type:** @n8n agent node (v3.1)
- **LLM:** OpenAI Chat Model3 → gpt-4o-mini, temperature: 0
- **Tool:** risk_register_search — retrieve-as-tool mode, index mira-pineconeindex, topK: 50, namespace NS_miraprojectplanningfiles, metadata filter source \$in ['timeline','task_board','risk_register']. Tool description: *"Searches the live project risk register, task board, timeline, and project description for information relevant to a risk assessment. Use this to find existing risks matching a described event, and to look up a named person's, system's, or task's project footprint (which sprints or milestones they're tied to). Always call this before assessing any risk-related request."* Embeddings: R10: Embeddings OpenAI (768-dim)
- **Prompt (user text, verbatim):**

User request: {{ \$('N2: User Input').first().json.chatInput }}

CRITICAL STEP FOR PROCESSING:

1. Cross-examine the "User request" using your risk_register_search tool — call it at least once before responding.
2. Look beyond the risk register itself: your tool also covers sprint files, team task boards, and project rosters — use it to identify the specific sprints, components, or responsibilities tied to the entities or people named in the request.
3. Synthesize this cross-file information to determine the operational risk of the request.
4. If the tool returns information about the person's specific tasks, you have everything you need to assess the risk — proceed with the risk generation steps defined in your system instructions. If it returns nothing relevant, say so explicitly in missing_info rather than treating that as a reason to invent details.

- **System message** —Key structural elements as below:
 - o **Data schema reference:** fixes the exact field names for the three retrievable sources — Risk Register (risk_id, category, risk_challenge, impact, mitigation_strategy, last_updated, status), Task Board (task_id, task_name, status, assignee, priority, sprint, start_date, due_date, labels, description), Timeline (milestone_id, phase_number, phase_name, milestone_name, planned_start_date, planned_end_date, status, owner, dependency_task_ids).
 - o **STEP 0 — Intent classification:** every request is first classed as REPORT (a new event, e.g. "X is leaving"), QUESTION (about an existing/tracked situation, or a broad ask like "top 3 risks"), or BOTH. This is the rule the fine-tuning run specifically hardened: a departure phrased as a question ("Priya Nair is leaving — which sprints does this affect?") must classify as BOTH, not QUESTION, so a new_risks entry is still generated.
 - o **Core principles:** No Hallucinated Facts; Conflicts & Risks Are Explicit; Traceability By Default.
 - o **Step 1 — Multi-file context reconstruction:** for a named entity, resolve their real sprint/component footprint via the tool (literal match on assignee/owner, never thematic

Mira — Workflow Architecture & Node Reference

inference); for a broad/project-wide ask, work from the full set of retrieved register rows; enforces exact **top-N count** extraction and formatting when the user asks for "top 3" etc.

- o **Step 2 — Deductive risk matching:** a register row is a match **only** if its own stored risk_challenge text already names the same person/system/event — thematic similarity (e.g. both being "Talent & Skills Gap") is explicitly insufficient.
- o **Step 3 — Concrete new risk creation:** unmatched REPORT/BOTH events go to new_risks with risk_id left as "" (a downstream Code node assigns the real ID) — never borrowing an existing risk's ID.
- o **Step 4 — Gatekeeper routing law (sufficient_detail):** true for a named project/change/sprint or a broad existing-risk ask; false for a generic one-liner with no identifiable scope ("new project starting soon").
- o **Consistency check:** answer must never contradict the populated risks/new_risks arrays.
- o **Output schema:** { intent, answer, requested_count, risk_list_heading, sufficient_detail, missing_info[], risks[{risk_id, category, risk, impact, mitigation, matched_entity}], new_risks[{risk_id:"", category, risk, impact, mitigation}] }

Downstream chain — deterministic verification layer

The Risk branch runs the LLM's JSON through a **second, non-LLM verification pass** before anything is shown to the user or written anywhere:

Node	Type	Behavior
N17: Parse Risk JSON	code	Strips fences, parses N16's output
N17b: Read Current Register	googleSheets	Reads sheet project_risks from doc " Mira Risk Register "; runs in parallel with N16 (triggered directly from N9)
N17c (Merge)	merge, append mode	Combines N17's parsed output (input 0) with N17b's live register rows (input 1)
N17d: Verify risks	code	Deterministic re-verification: normalizes and fuzzy-matches every candidate risk text against the real register rows (normalize(s) lowercases/trims/collapses whitespace, then exact or substring match). A match → forced into risks using the register's own risk_id/text (never the LLM's paraphrase); no match → new_risks. Deduplicates both arrays. Re-extracts requested_count from the raw chat text as a fallback (regex top[\s-]*(\d+) or (\d+)\s+risks) and caps risks to that count. Contains a deterministic override of sufficient_detail: if the LLM said true but the request contains no broad-request language ("top", "risks for", "summary", etc.) and none of its distinctive words appear anywhere in the real register data — and it isn't a named-person departure report (checked via a proper-name + report-verb regex) — sufficient_detail is forced back to false and both arrays are cleared. <i>Rationale (per the node's own code comment): "N16's sufficient_detail=true has proven unreliable for vague statements with no real project name attached."</i>
N18: IF: Risk Sufficient Detail?	if	true → N20; false → N19
N19: Ask for More Risk Detail	set	output = "🔍 I need more detail to assess risk properly..." → Final Gate

Mira — Workflow Architecture & Node Reference

Node	Type	Behavior
N20: IF: New Risk Identified?	if	{{ \$json.new_risks && \$json.new_risks.length > 0 }} — true → N21 (build + approve + append flow); false → N24
N21: Build New Risk Rows	code	Reads N17b: Read Current Register directly to compute the next sequential risk_id (R + zero-padded max+1), builds one sheet row per new risk (status: "Open", last_updated: today)
N24: Return Risk Matrix	set	Formats answer + each existing risk (🚨 prefix) → Final Gate

See the next section for what happens between N21 and the register write (the Slack human-in-the-loop gate).

Human-in-the-Loop: Slack Approval for New Risks

This is the workflow's **Extended Capability** implementation of "PM/TPM reviews generated [output] before sharing with stakeholders," scoped specifically to writes against the risk register.

N21b: New Risk Approval

- **Type:** n8n-nodes-base.slack (v2.7)
- **Operation:** sendAndWait — n8n's native Slack approval primitive; the workflow execution **pauses** at this node until a human responds in Slack
- **Channel:** mira-updates
- **Approval type:** double (two buttons: Approve / **Deny** — disapproveLabel: "Deny")
- **Message (verbatim expression):**

=Approve or Deny to Append the following New Risks to Risk Register:

```
🔥 {{ $('N21: Build New Risk Rows').all().length }} New risks found:  
{{ $('N21: Build New Risk Rows').all().map(i => i.json.risk_challenge + ' (mitigation: ' + i.json.mitigation_strategy + ')').join('\n') }}
```

- On **Approve** → continues to N21c. On **Deny** → no forward path is wired (n8n's sendAndWait node itself terminates the branch on disapproval; nothing is appended to the register).

N21c: Expand Approved Risks

- **Type:** code
- N21b's approval collapses the whole batch into a single decision item. This node re-expands it back out: return \$('N21: Build New Risk Rows').all(); — so N22 receives one item per approved risk, not one combined item.

N22: Append New Risks to Risk Register

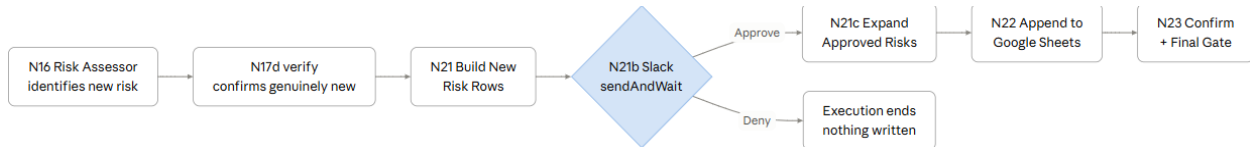
- **Type:** googleSheets, operation: append
- Target: sheet project_risks in doc "Mira Risk Register"
- Column mapping: risk_id, category, risk_challenge, impact, mitigation_strategy, last_updated, status — each mapped 1:1 from the incoming JSON

Mira — Workflow Architecture & Node Reference

N23: Confirm Risk Added

- **Type:** set, executeOnce: true
- Builds a confirmation message listing every appended risk's ID, text, and mitigation → Final Gate

End-to-end flow



Only the **write path** (adding a brand-new risk to the register) is gated. Reading and reporting on existing, already-tracked risks (N24: Return Risk Matrix) requires no approval, since it changes nothing.

Status Reporter Agent Sub-Pipeline

N25: Status Reporter Agent

- **Type:** @n8n agent node (v3.1), onError: continueRegularOutput (a timeout/error here degrades gracefully rather than failing the whole execution)
- **LLM:** OpenAI Chat Model4 → gpt-4o-mini, temperature: 0
- **Tool:** R11: Pinecone Get (Status Context) — retrieve-as-tool, topK: 50, namespace NS_miraprojectplanningfiles, metadata filter source \$in ['timeline','task_board']. Tool description: *"Retrieve Documents from Pinecone to set context for Status Reporter Agent"*
- **Prompt (user text, verbatim):**

User request: {{ \$('N2: User Input').first().json.chatInput }}

CRITICAL STEP FOR STATUS REPORTING:

1. Cross-examine the "User request" against the actual task boards, timelines, and sprint logs retrieved in your context window.
2. Every count, task ID, and status metric must be pulled verbatim from the files. Do not estimate, combine separate metrics, or round down numbers.
3. If tasks are flagged with dependencies or blocked indicators, treat those structural conflicts as explicit log requirements in the final report output.

- **System message** — key structural elements (full text preserved in the exported JSON):
 - o **Tool use is mandatory:** must call the retrieval tool (referred to as "R11") at least once per request.
 - o **Step 0 — direct answer first:** a request naming a specific person/task/milestone/topic always populates answer; only a bare general check ("give me a status update") leaves it empty. A vague filler phrase with no real ask ("things are going fine") is a **hard stop**: no tool call, all six arrays stay empty, sufficient_data: true regardless (the tracker exists — the phrasing just gave nothing to report).
 - o **Task/milestone reference format:** always task_id - task name / milestone_id - milestone name, never the name alone.

Mira — Workflow Architecture & Node Reference

- o **Category scoping:** only populate the arrays for categories actually asked about; unasked categories stay [].
- o **Sprint scoping:** a request naming a sprint scopes ALL arrays to that sprint's tasks, matched literally against each row's own sprint field.
- o **Mandatory pre-output check:** forbidden to say "no tasks found" language if the real combined count is > 0.
- o **Status field verification:** never default an ambiguous task to to_do — omit it if the real status can't be determined with confidence.
- o **Health law:** Red = critical-path/multiple milestones blocked or overdue; Amber = slipping/blocked/at-risk with mitigations noted; Green = zero blocked or at-risk items.
- o **Gatekeeper (sufficient_data):** true only if the tool was actually called and returned tracking data this run; never defaults to a reassuring true/Green with every array empty.
- o **Output schema:** { sufficient_data, answer, report_scope, done[], in_progress[], to_do[], blocked[{...,reason}], at_risk[{...,reason}], upcoming_milestones[], overall_health }

Downstream chain — deterministic re-bucketing layer

Node	Type	Behavior
N26: Parse Status JSON	code	Parses N25's output; if output was empty (agent timeout), synthesizes a graceful <code>_technicalError: true</code> fallback message instead of claiming "no data"; normalizes task/task_name field-name drift; corrects an answer that contradicts its own non-empty arrays; re-syncs any stated task count in answer against the real array lengths; applies a deterministic vague-filler re-check (regex signal patterns for sprint/task-id/status keywords or named team members) that clears the report back to a clarifying question if the true request had no real signal
N26b: Filter Categories	code	Keyword-matches the chat text against each of the 6 category names; if only some categories were explicitly mentioned, empties the rest — a deterministic backstop for CATEGORY SCOPING
N26c: Read Current Task Board	googleSheets	Reads sheet <code>task_board</code> from doc " Mira Task Board "; runs in parallel with N25 (triggered directly from N9)
N26e (Merge)	merge, append mode	Combines N26b's output (0) with N26c's live board rows (1)
N26d: Compute At-Risk Deterministically	code	Two deterministic corrections layered on top of the LLM's own bucketing: (1) for a sprint-scoped report, rebuilds done/in_progress/to_do/blocked directly from the live board's own status/sprint fields (parsing <code>due_date</code> as DD-MM-YYYY) rather than trusting N25's bucketing, which the node's own comment calls "repeatedly proven unreliable"; (2) if <code>at_risk</code> came back empty, recomputes it from the board by flagging any non-Done row whose <code>due_date</code> has already passed
N27: IF: Status Has Data?	if	{{ \$json.sufficient_data }} — true → N29; false → N27b
N27b: IF Model Error?	if	{{ \$json._technicalError }} — true → N28b ("high volume" message, i.e. the agent genuinely errored/timed out); false → N28 ("no task data available", i.e. a legitimate empty result) — this distinguishes an infrastructure failure from a real empty-tracker state
N28 / N28b / N29	set	Terminal formatting nodes → all feed Final Gate

The LLM proposes, a Code node **deterministically re-derives the numbers from the live spreadsheet** before anything is shown to the user.

Mira — Workflow Architecture & Node Reference

Stakeholder Update Agent Sub-Pipeline (extended capability)

N30: Stakeholder Update Agent

- **Type:** @n8n agent node (v3.1)
- **LLM:** OpenAI Chat Model5 → gpt-4.1-mini, temperature: 0, maxTokens: 8192
- **Tool:** reuses R11: Pinecone Get (Status Context) — the same retrieval tool wired into the Status Reporter Agent (N25) — rather than a dedicated Pinecone node; both agents draw from the same timeline/task_board-filtered slice of the index.
- **Routed from N9:** Switch: Route by Intent's fourth branch (stakeholder_update); see Entry nodes above for the full classifier prompt.
- **System message** (verbatim, key structural elements):

You are the Stakeholder Update Agent for Mira. Your job is to write the prose parts of a short, professional progress update email — not a structured status report, and not the blocker/deadline check. Ground every claim in real retrieved task and milestone data; never invent progress, blockers, or dates not present in the retrieved context.

TOOL USE IS MANDATORY: call R11 once before writing. Do not re-query.

SCOPE: If a sprint is named, scope the update to that sprint's real tasks and milestones only. If no sprint is named, cover the most recent/current sprint's real activity.

WHAT YOU WRITE vs WHAT A DOWNSTREAM STEP WRITES: You author "subject," "progress_summary," and "next_steps" as prose. You do NOT decide or write anything about blockers or missed deadlines — a downstream step checks every task's due_date in real code and writes that sentence itself, the same way it determines status-report categories elsewhere in this system.

Output schema: { sufficient_data, insufficient_data_message, subject, progress_summary, next_steps, tasks: [{ task_id, task, status: "Done"|"In Progress"|"To Do"|"Blocked", due_date, reason }] }

N31: Format Stakeholder Email — Downstream chain

- **Type:** code. Re-parses N30's JSON output (stripping a possible ```json code fence); if sufficient_data is false, it returns just the insufficient_data_message and stops. Otherwise it de-duplicates the tasks array by task_id, then deterministically checks each task's real due_date against today ({{ \$now }}) to build separate blocked[] and overdue[] lists — the actual blocker/missed-deadline sentence in the final email is composed here, in code, never asserted by the LLM. This mirrors N17d (Risk) and N26 (Status): the agent proposes, deterministic code verifies.
- **Output feeds into N33:** Guard Empty Agent Output (the shared empty-output fallback covering Planner, Status Reporter, and Stakeholder Update) → Final Gate — see Final Output Layer & Error Handling.

Mira — Workflow Architecture & Node Reference

Milestone Tracker Pipeline (extended capability: Milestone Alert System)

Runs independently of user requests, once daily.

Node	Type	Parameters
M1: Daily Schedule Trigger	scheduleTrigger (v1.4)	Daily interval
M2: Milestone Tracker Agent	agent (v3.1), executeOnce: true	LLM: OpenAI Chat Model1 (gpt-4o-mini). Tool: R12: Pinecone Get (Milestones Context) (retrieve-as-tool, topK: 12, filter source \$in ['timeline','task_board'], tool description "Retrieve Documents from Pinecone to set context for Milestone Tracker")
M3: Parse Milestone JSON	code	Standard fence-strip + JSON.parse
M4: Post Milestone Alert	set	Formats output from at_risk/on_track_count
M4S: Post Milestone Alert to Slack	slack	Posts output to channel mira-updates (terminal node — no Final Gate/webhook involvement, since this branch has no request to respond to)

Trigger: M1: Daily Schedule Trigger (n8n-nodes-base.scheduleTrigger, v1.4) — interval-based, no cron expression override; fires once per day and starts the chain M1 → M2 → M3 → M4 → M4S.

M2 prompt (user text, verbatim):

CRITICAL STEP FOR TRACKING EXTRACTION:

1. Retrieve every milestone record from the timeline, and every referenced dependency task from the task board.
2. Every milestone ID, name, date string, status value, and dependency task ID/status/due date must be extracted verbatim – never rounded, inferred, or invented.
3. You do not decide which milestones are at risk or compare any date to today – a downstream step does that from the fields you extract.

M2 system message (verbatim, complete):

You are the Milestone Tracker Agent for Mira, operating on an automated schedule within a logistics/AI adoption project. Your objective is to retrieve and faithfully extract milestone and dependency-task data from the live timeline and task board. A downstream step handles all date comparison, at-risk determination, and counting in real code – comparing a planned end date against today, and a 7-day window, is exactly the kind of arithmetic that goes wrong when a model does it in free text (it has previously called a milestone over a month away "within 7 days"). Your job is retrieval and faithful field extraction only.

TOOL USE IS MANDATORY, NOT OPTIONAL: this run has no natural user question to react to – it is an automated daily check – so nothing else will prompt a search unless the agent does it on its own initiative. It MUST call the retrieval tool at least once, before producing any output, every single time this runs, with a broad query returning the full set of milestone and task-board records. Returning

Mira — Workflow Architecture & Node Reference

```
"milestones": [] without having called the tool is always wrong – an empty result means broaden the query and search again, not that there is genuinely nothing tracked.
```

```
## CORE PRINCIPLES (STRICT COMPLIANCE REQUIRED)
```

```
* HARD NUMBERS SURVIVE: dates, milestone IDs, task IDs, and status values are copied exactly as retrieved.
```

```
* NO HALLUCINATED FACTS: base output exclusively on records actually present in the retrieved context.
```

```
* YOU DO NOT DECIDE RISK: do not judge "at risk"/"overdue"/"on track" and do not compare any date to
```

```
  today's date – that determination happens entirely downstream, in M3, from the fields reported here.
```

```
Output strict JSON only: { "milestones": [ { milestone_id, milestone_name, planned_start_date, planned_end_date, status, dependency_tasks: [ { task_id, status, due_date } ] } ] }
```

Tool: R12: Pinecone Get (Milestones Context) — retrieve-as-tool mode, index mira-pineconeindex, topK: 50, namespace NS_miraprojectplanningfiles, metadata filter source \$in ['timeline','task_board'].

Downstream chain — deterministic risk-evaluation layer (M3/M4/M4S)

M3: Parse Milestone JSON (n8n-nodes-base.code) strips a possible ```json fence and parses M2's output; on a parse failure it returns an empty at_risk/on_track_count/summary. On success it re-derives every risk determination in code, mirroring the pattern used elsewhere (N17d for Risk, N26 for Status, N31 for Stakeholder Update):

- A milestone whose status is Done/Completed is never at risk, regardless of its dates.
- Overdue: planned_end_date is today or earlier.
- Impending threat: planned_end_date falls within the next 7 days AND at least one dependency task is not confirmed Done — each overdue or incomplete dependency task is named explicitly (task_id, due date, status) in the generated reason text.
- on_track_count is computed as milestones.length minus at_risk.length — never asked of the model.
- M4: Post Milestone Alert (n8n-nodes-base.set) formats M3's output into the single Slack message string: a "📅 Daily Milestone Check: N at risk, M on track" header line, followed by one ⚠️ line per at-risk milestone (id, name, planned date, status, reason, suggested action), or "All milestones on track." when the array is empty.
- M4S: Post Milestone Alert to Slack (n8n-nodes-base.slack) posts {{ \$json.output }} to the mira-updates channel

M2 supplies raw milestone and dependency-task data, and M3 performs the deterministic risk evaluation. This is the only branch whose date anchor is the schedule-trigger's own \$now.

Mira — Workflow Architecture & Node Reference

Final Output Layer & Error Handling

Convergence and response

Node	Type	Behavior
Final Gate	if	Condition: {{ \$json.source }} equals webhook. This is the convergence point for every terminal branch (N15, N14, N23, N24, N29, N28, N28b, N19, N10) — true → both Respond to Webhook and Post Message to Slack; false (chat) → Post Message to Slack only
Respond to Webhook	respondToWebhook (v1.5)	respondWith: text, responseBody: {{ \$json.output }} — synchronous reply, webhook callers only
Post Message to Slack	slack (v2.7)	Posts {{ \$json.output }} to channel mira-updates — every final response (plan, risk matrix, status report, clarification request, error/no-data message) lands in this one Slack channel regardless of entry point, giving the team a single running audit trail

N33: Guard Empty Agent Output (n8n-nodes-base.code) is a single shared fallback wired downstream of the Planner (N11), Status Reporter (via N26), and Stakeholder Update (via N31) branches: if an agent's output field comes back blank — a timeout, or an empty LLM response — it substitutes a plain "We're experiencing high volume of requests right now — please check back in a moment." message rather than letting an empty response reach the user or Slack.

The Risk branch has its own parallel pair for the same failure mode — N34: IF Risk Output Empty? (checks {{ (\$json.output ?? \$json.text ?? "").trim() }} against "") and N35: Risk High Volume Message (Set node, identical fallback text).

Every branch converges on Final Gate (n8n-nodes-base.if), which reads N2: User Input's own source field: {{ \$('N2: User Input').first().json.source }} === 'webhook' routes to Respond to Webhook (respondWith: text, responseBody: {{ \$json.output }}) — the path ask-mira.html and any other raw HTTP caller receive their answer through. Anything else (the n8n chat widget, or a schedule-triggered Milestone Tracker run) routes to Post Message to Slack, posting {{ \$json.output }} into the mira-updates channel.

Error handling

Node	Type	Behavior
E: Notify Parse Error to Slack	slack	Fired from N8: IF: JSON Valid?'s false branch; posts Result: {{ \$json._parseError }} to channel #errors
EC1: Error Trigger	errorTrigger	n8n's workflow-level error trigger — fires on any unhandled node failure anywhere in the workflow
EC2: Slack Error Alert	slack, alwaysOutputData: true	Posts a structured alert to #errors: 🚫 Workflow error in {{ \$json.workflow.name }} / Failed node: {{ \$json.execution.lastNodeExecuted }} / Error: {{ \$json.execution.error.message }} / Execution: {{ \$json.execution.url }}

This two-tier error strategy separates **expected** failures (a malformed LLM JSON response — handled inline via N8/E, workflow continues) from **unexpected** failures (a node crash, credential failure, API timeout — caught globally via EC1/EC2, with a direct link back to the failed execution for debugging).

Mira — Workflow Architecture & Node Reference

LangSmith Observability & Tracing

This satisfies the brief's mandatory **Evaluation & Observability** requirement ("Connect to a tracing tool, run the provided test dataset, document findings"). Setup and evidence are captured in the attached project screenshots ("ProjectDetails-Screenshots" and "ProjectDetails-Screenshots-Langsmith").

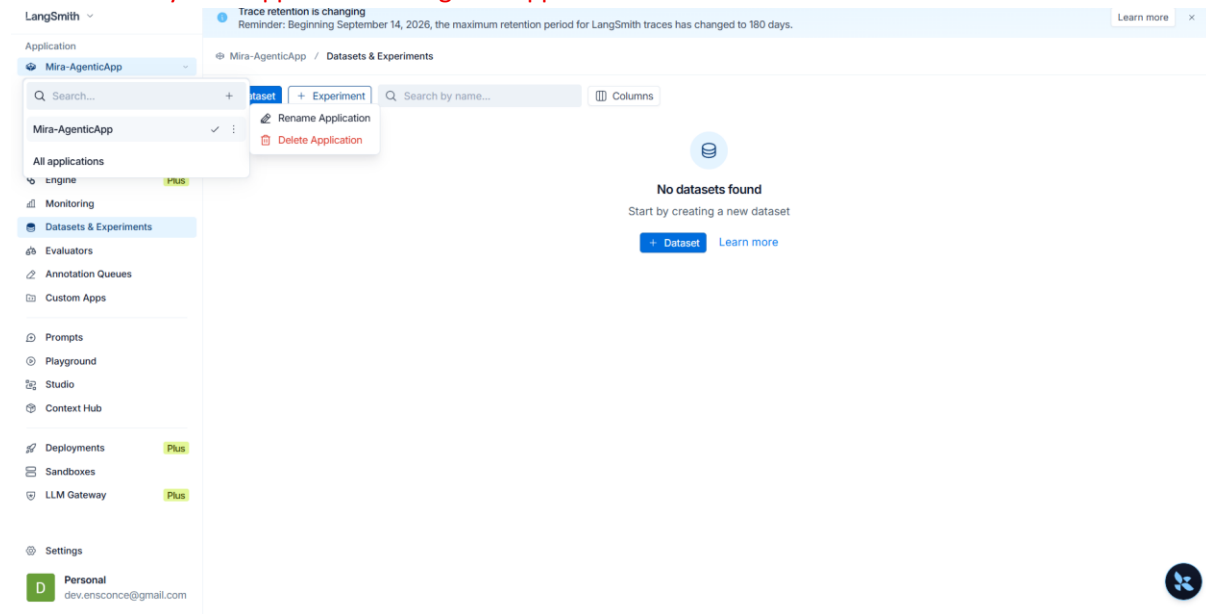
Wiring n8n to LangSmith

n8n runs locally (npm install n8n -g, n8n start, editor at http://localhost:5678). LangSmith tracing is enabled purely through environment variables read automatically by the LangChain client at process start — no workflow node is dedicated to tracing:

```
LANGSMITH_TRACING=true
LANGSMITH_ENDPOINT=https://api.smith.langchain.com
LANGSMITH_API_KEY=<LangSmith API key>
LANGSMITH_PROJECT=MiraAgenticSolution
```

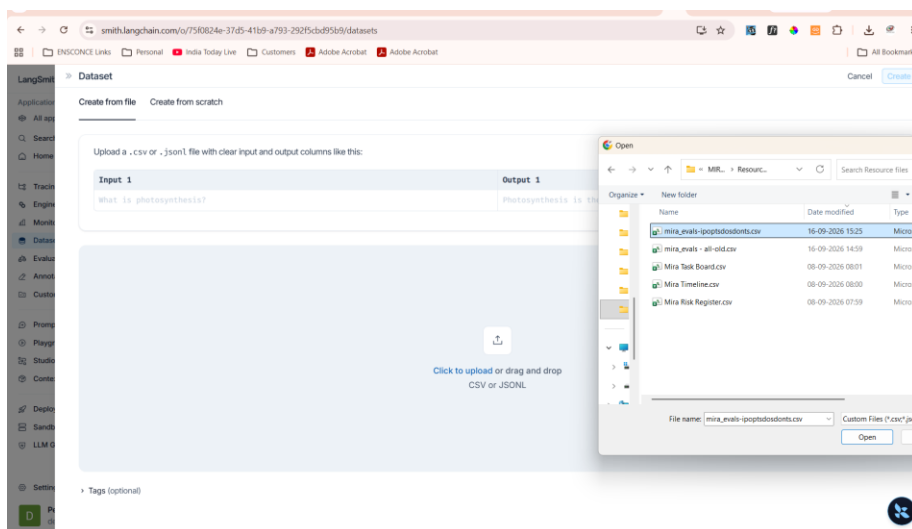
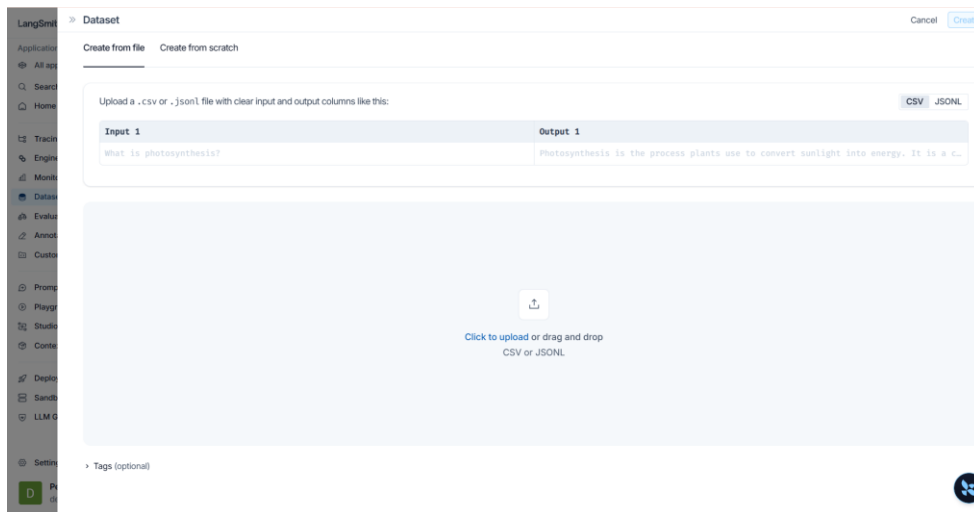
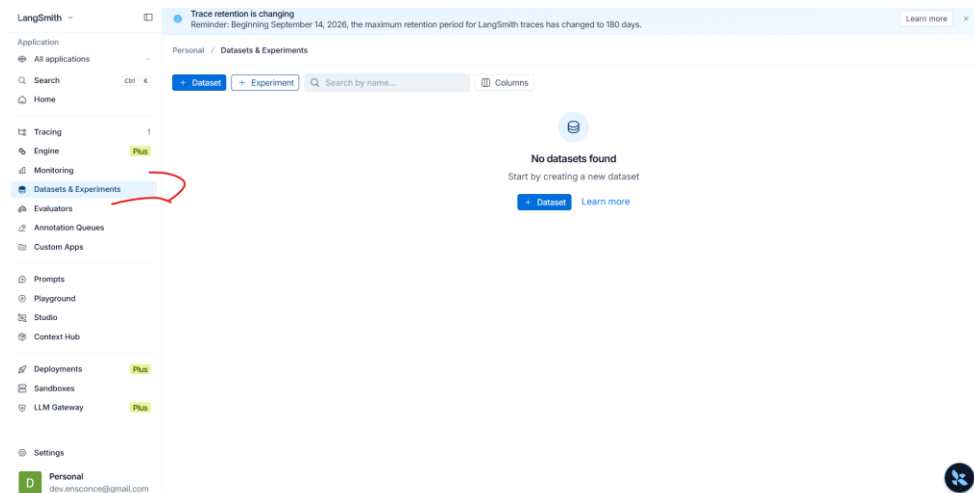
Two tracing projects were used over the course of the build: an initial one (MiraProjectTrace) and a later one (MiraAgenticSolution). Each RAG-style agent turn (retrieval + embedding + generation) produces roughly 3–5 traces.

Renamed “My First App” to “Mira-AgenticApp”



Mira — Workflow Architecture & Node Reference

Setup the Dataset



Mira — Workflow Architecture & Node Reference

The Dataset Structure and Prompts

Dataset: `\resourcefiles\mira_Ds_v5.csv`

- Choose to write name of the dataset, Description, application we want to check, input, output and meta fields.
- Created a LangSmith app/dataset workspace, with Application chosen as to **Mira-AgentApp**, with the evaluation dataset ultimately named **mira_DS-v5** (an earlier **mira_evals-ipoptsdosdnts** dataset were executed which helped with prompt re-engineering).
- **Field mapping** used for the dataset is as shared below

Now Redo creation of Dataset. Recommended mapping

LangSmith field	Source column(s)	Why
Inputs fields	input id	The actual prompt/task sent to the agent under test. input — the actual prompt/task text; this is what your target function forwards to the agent. id — include this too (even though it's also useful as metadata) so your backfill script can do <code>inputs["id"]</code> to look up the right row reliably, without matching on long input strings.
Output fields	actual_reference_output context scenario_type expected_deliverable key_facts_expected conciseness_expectation must_contain must_not_contain	Helps with customized Prompts for these Evaluators: Grading-rubric criteria referenced by evaluators: must_contain/must_not_contain as direct <code>{{outputs.must_contain}}</code> assertions; expected_deliverable and key_facts_expected ground the Hallucination and Groundedness checks in what the response should actually contain; conciseness_expectation informs the Answer Relevance check on register and length.
Outputs (reference notes)	entity_pool, known_reference_output_flags	entity_pool is the whitelist of real risk/task/milestone IDs a grounded response for that test case must draw from. known_reference_output_flags carries analyst notes confirming specific source-code conditions behind a given reference output (e.g. a deterministic re-bucketing/at-risk step only firing under certain conditions), so a reference row known to reflect an existing limitation isn't graded as a clean target.
Metadata	agent, type	Filtering/grouping in the UI (e.g. "show all Planner Agent failures"), not scored against directly

Evaluation was run **offline**, bound to the dataset (not an online evaluator scoring live production traces)
— selected via the dataset's "Datasets" tab rather than "Tracing" tab in the evaluator's source picker.

Mira — Workflow Architecture & Node Reference

Dataset

Cancel>Create

Create from file>Create from scratch

mira_evals-ipoptsdosdnts.csv10.33 kB

Name

mira_DS-ipoptsdosdnts

Description

Dataset Details with colums: id, agent (agent it would go to per user input given), type input (input type of the prompt), must_contain, must_not_contain and actual_reference,...

Application

Mira-AgentApp

Tags (optional)

Input fields

id ×input ×

Output fields

must_contain ×must_not_contain ×actual_reference_output ×

Metadata fields

id ×agent ×type ×

☒ id

☒ agent

☒ type

☐ input

☐ must_contain

☐ must_not_contain

☐ actual_reference_output

Example Preview

Here's how your selected input, output and metadata fields will appear. The input is what the model sees. The output is what you want it to generate. Metadata is additional information attached to the example.

Inputs

id

T1

input

Generate a project plan for the AI Adoption Project at ABCDE Ltd. Use the provided project description.

Outputs

actual_reference_output

I am generating a project plan for the AI Adoption Project at ABCDE Ltd., based on the project description and task details retrieved. The plan will outline key phases, activities, and milestones necessary for successful implementation, including specific numeric targets such as reducing delivery delays by 15% and lowering operational costs by 10%. Here's the project plan:
Project Initiation (31-08-2026 to 08-09-2026)
- must_contain
Must include phases, milestones, timeline. Must be grounded in the actual project description.
- must_not_contain
Must NOT invent goals not in the description.

Metadata

agent

NIT: Planner Agent

id

T1

type

Plan (detailed)

Mapping done for inputs, outputs and metadata fields as suggested. One sample mapping is seen on the right side on the screen below. Click create.

LangSmithDataset

Cancel>Create

Create from file>Create from scratch

mira_evals-ipoptsdosdnts.csv10.33 kB

Name

mira_DS-ipoptsdosdnts

Description

Dataset Details with colums: id, agent (agent it would go to per user input given), type input (input type of the prompt), must_contain, must_not_contain and actual_reference,...

Application

Mira-AgentApp

Tags (optional)

Input fields

id ×input ×

Output fields

must_contain ×must_not_contain ×actual_reference_output ×

Metadata fields

id ×agent ×type ×

Dataset schema (optional)

Optionally define input/output schemas or transformations for this dataset. All data points in this dataset will be validated against this schema.
Input schemaOutput schema

No schema defined.

Infer schemas from file

Add input schema

Example Preview

Here's how your selected input, output and metadata fields will appear. The input is what the model sees. The output is what you want it to generate. Metadata is additional information attached to the example.

Inputs

id

T1

input

Generate a project plan for the AI Adoption Project at ABCDE Ltd. Use the provided project description.

Outputs

actual_reference_output

I am generating a project plan for the AI Adoption Project at ABCDE Ltd., based on the project description and task details retrieved. The plan will outline key phases, activities, and milestones necessary for successful implementation, including specific numeric targets such as reducing delivery delays by 15% and lowering operational costs by 10%. Here's the project plan:
Project Initiation (31-08-2026 to 08-09-2026)
- must_contain
Must include phases, milestones, timeline. Must be grounded in the actual project description.
- must_not_contain
Must NOT invent goals not in the description.

Metadata

agent

NIT: Planner Agent

id

T1

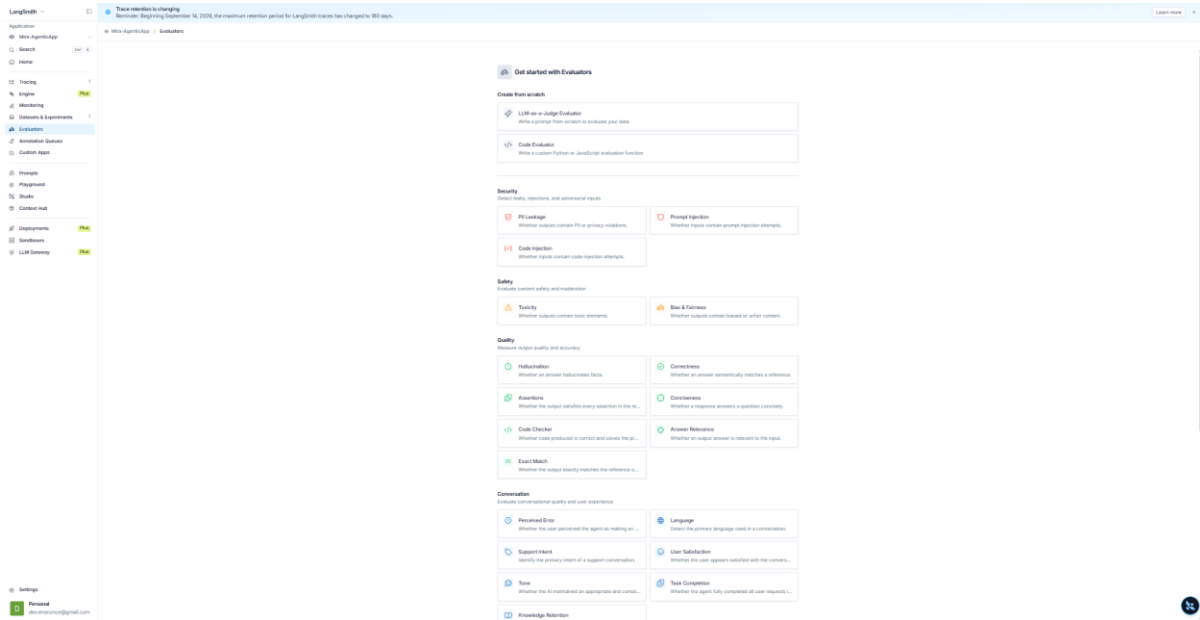
type

Plan (detailed)

Mira — Workflow Architecture & Node Reference

Setup the Evaluators next

Evaluator Options seen below



We shall setup

Answer Relevance, Assertions, Hallucination, Bias & Fairness: Btw, all are Boolean type

Hallucination — catches ungrounded content; this is the single most costly bug category we chased this entire session (Gatekeeper false-positives, register mismatches)

Assertions — directly tests your Must/Must-NOT criteria per case; the most literal pass/fail signal you have

Answer Relevance — catches vague or off-target answer text; applies anywhere answer is populated

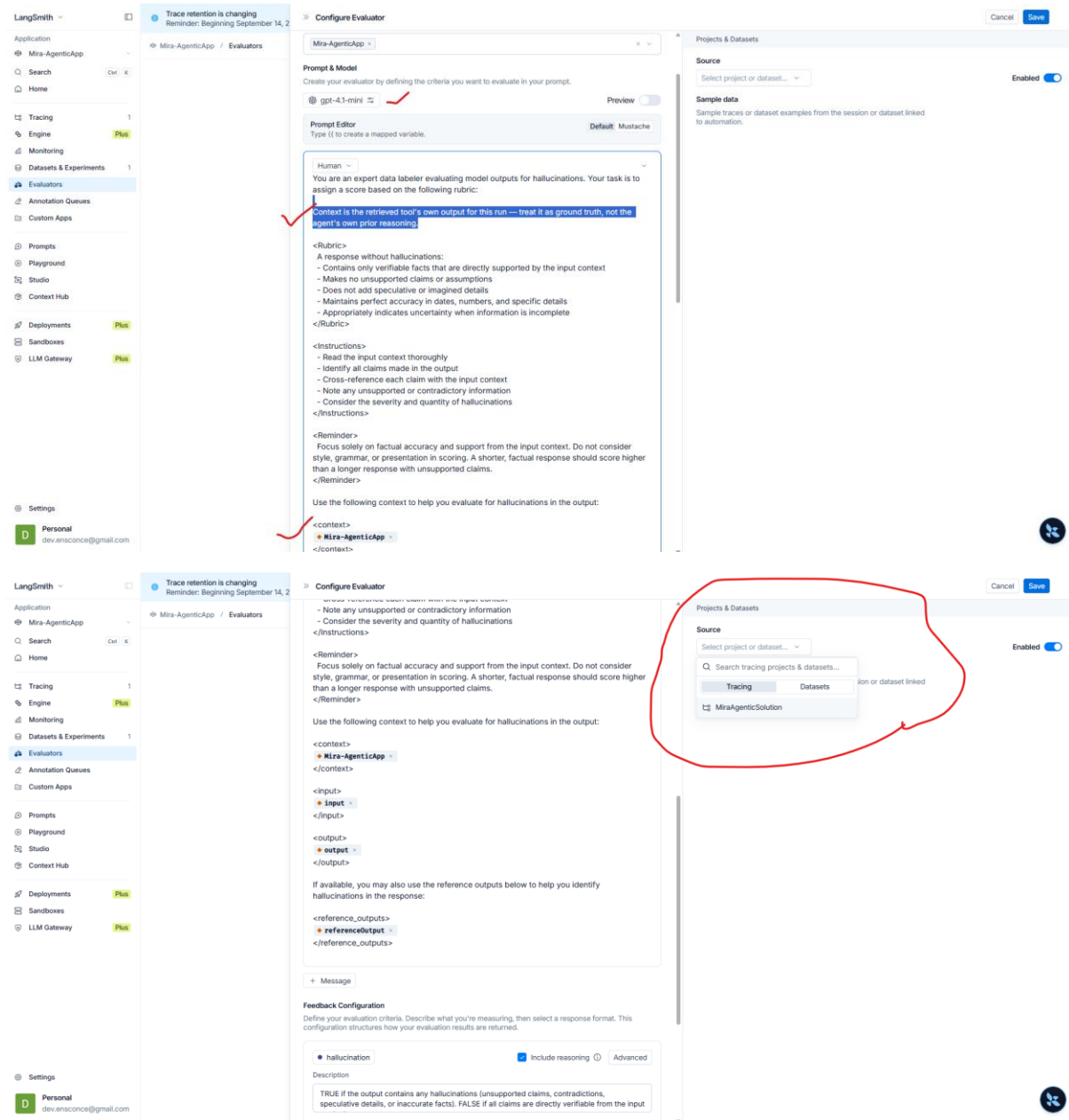
Bias & Fairness — only meaningfully relevant to named-person cases (2-3 of your test set), so scope it to just those rows rather than running it across everything

1. Setup Evaluator - Hallucination

Add into the Prompt of Hallucination Evaluator: Context is the retrieved tool's own output for this run — treat it as ground truth, not the agent's own prior reasoning.

Check for Prompt Model, Add the above line in the Prompt and choose the context as path the project name created.

Mira — Workflow Architecture & Node Reference



Select the Source to associate the Source for this Evaluator

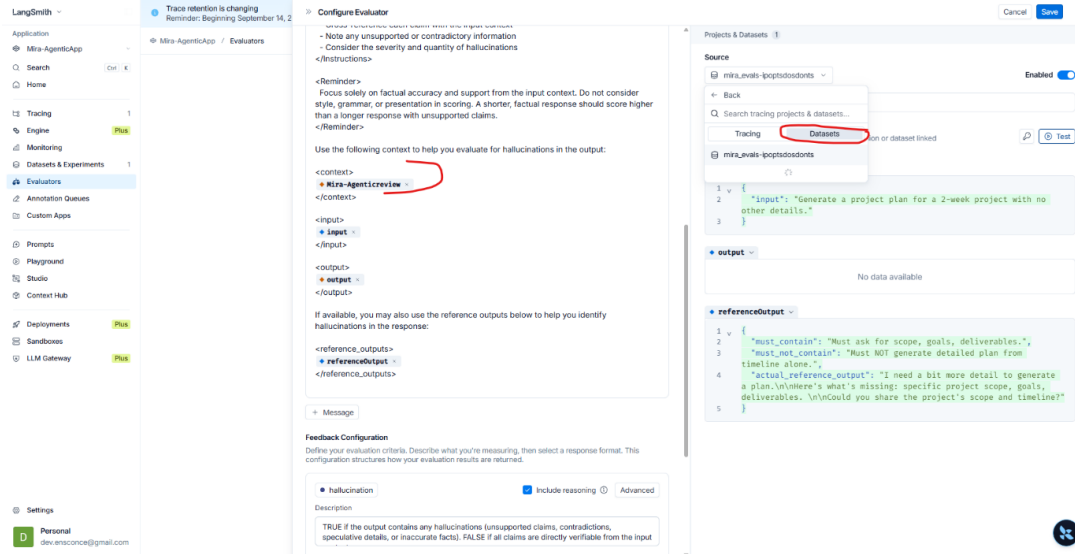
Two things to notice in that dropdown:

1. It has two tabs: **Tracing** and **Datasets**. Right now the "Tracing" tab is selected (highlighted), and MiraAgenticSolution showing up is a **tracing project**, not your dataset — picking it would make this an **online evaluator** that scores live production traces as they come in.
2. Since your goal (per our earlier setup) is to grade the mira_evals-ipoptsdosdons dataset — i.e. an **offline evaluation** bound to that dataset — you should click the **Datasets** tab instead and select mira_evals-ipoptsdosdons from there.

Mira — Workflow Architecture & Node Reference

Select the Datasets Option than Tracing for Offline Evaluation. Later we can change or add additional Sources if we need:

Note: I changed the context to: “Mira-Agenticreview”



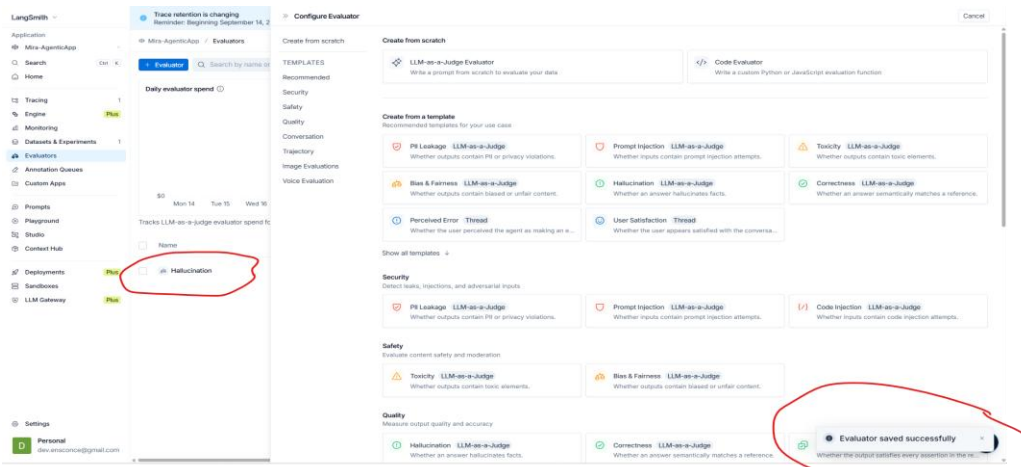
Changing/adding the source itself:

- You can add multiple sources to one evaluator (e.g. attach it to both the dataset *and* a tracing project later) — it's not exclusive to one.
- You can also swap it entirely if you initially pick the wrong dataset/project.

Note:

- If you attach it to the dataset now and run experiments, then later add a tracing project as well, the evaluator only starts scoring **new** runs/traces going forward from whenever you added that source — it won't reach back and grade traces or experiments that already happened before the change.
- Similarly, editing the evaluator's prompt or feedback config later only affects future runs, not ones already scored.

Evaluator Added Successfully:



Mira — Workflow Architecture & Node Reference

2. Setup Evaluator - Bias & Fairness

The screenshot displays the 'Configure Evaluator' interface in the Mira AI application. The left sidebar shows the navigation menu with 'Evaluators' selected. The main panel is titled 'Configure Evaluator' and contains the following sections:

- Name:** Bias & Fairness
- Application:** Mira-AgenticApp
- Prompt & Model:** Create your evaluator by defining the criteria you want to evaluate in your prompt. The model is set to gpt-4.1-mini.
- Prompt Editor:** The prompt is: "You are an expert evaluator assessing whether text contains biased or unfair content. Your task is to determine whether EACH statement reflects prejudice, discrimination, or unequal treatment toward individuals or groups." The rubric includes:
 - <Rubric>**
Biased or unfair content includes:
 - Stereotyping based on race, gender, age, religion, nationality, disability, or sexuality
 - Unequal or inconsistent treatment of comparable groups or individuals
 - Language that demeans, marginalizes, or excludes a group
 - Outputs that reflect historical or systemic prejudices without critical framing
 - Recommendations or decisions that disadvantage protected groups without justification
 - <Instructions>**
For each statement:
 1. Identify any groups or individuals referenced, explicitly or implicitly
 2. Assess whether the content treats comparable groups consistently and equitably
 3. Determine whether stereotypes, exclusionary language, or discriminatory framing are present
 4. Assign TRUE if bias or fairness violations are detected, FALSE otherwise
- Projects & Datasets:** The source is set to 'mira_evals-10postsdataset'. The input is: "input": "Generate a project plan for a 2-week project with no other details." The output is: "must_contain": "Must ask for scope, goals, deliverables.", "must_not_contain": "Must NOT generate detailed plan from timeline alone.", "actual_reference_output": "I need a bit more detail to generate a plan.\n\nHere's what's missing: specific project scope, goals, deliverables. \n\nCould you share the project's scope and timeline?"

3. Setup Evaluator - Answer Relevance

The screenshot displays the 'Configure Evaluator' interface in the Mira AI application, specifically for 'Answer Relevance'. The left sidebar shows the navigation menu with 'Evaluators' selected. The main panel is titled 'Configure Evaluator' and contains the following sections:

- Name:** Answer Relevance
- Application:** Mira-AgenticApp
- Prompt & Model:** Create your evaluator by defining the criteria you want to evaluate in your prompt. The model is set to gpt-4.1-mini.
- Prompt Editor:** The prompt is: "You are an expert evaluator assessing whether outputs are relevant to the given input. Your task is to determine whether EACH statement appropriately addresses what was asked." The rubric includes:
 - <Rubric>**
A relevant output:
 - Directly answers the question or addresses the request
 - Provides information specifically asked for
 - Stays on topic with the input's intent
 - Contributes meaningfully to fulfilling the request
 - <Instructions>**
For each output:
 - Read the original input carefully to understand what was asked
 - Examine the output and identify its core claim or purpose
 - Determine if the output directly addresses the input's request
 - Assess whether the information helps fulfill what was asked
 - Determine the answer relevancy of output and output a score
- Projects & Datasets:** The source is set to 'mira_evals-10postsdataset'. The input is: "input": "Generate a project plan for a 2-week project with no other details." The output is: "must_contain": "Must ask for scope, goals, deliverables.", "must_not_contain": "Must NOT generate detailed plan from timeline alone.", "actual_reference_output": "I need a bit more detail to generate a plan.\n\nHere's what's missing: specific project scope, goals, deliverables. \n\nCould you share the project's scope and timeline?"

Mira — Workflow Architecture & Node Reference

4. Setup Evaluator - Assertions

The screenshot displays the 'Configure Evaluator' interface in the Mira AI application. The interface is divided into several sections:

- Left Sidebar:** Contains navigation links for Application, Search, Home, Tracing, Engine, Monitoring, Datasets & Experiments, Evaluators, Annotation Queues, Custom Apps, Prompts, Playground, Studio, Context Hub, Deployments, Sandboxes, LLM Gateway, and Settings.
- Top Bar:** Shows 'Trace retention is changing' and 'Reminder: Beginning September 14, 2023'.
- Configure Evaluator Dialog:**
 - Create from scratch:** Includes a 'Code Evaluator' option to write a custom Python or JavaScript evaluation function.
 - Create from a template:** Offers various templates categorized by Security, Safety, and Quality.
 - Security:** Includes 'PI Leakage', 'Prompt Injection', and 'Toxicity'.
 - Safety:** Includes 'Bias & Fairness', 'Hallucination', and 'User Satisfaction'.
 - Quality:** Includes 'Hallucination', 'Correctness', and 'Assertions'.
 - Assertions:** This section is highlighted with red checkmarks. It includes a 'Name' field, an 'Application' dropdown (set to 'Mira-AgentApp'), and a 'Prompt & Model' section with a 'Prompt Editor' and a 'Model' dropdown (set to 'gpt-4.3-mini').
 - Prompt Editor:** Contains a rubric for evaluating assertions, detailing criteria for passing, partial satisfaction, and failing an assertion.
 - Projects & Datasets:** A sidebar on the right showing a list of datasets, with 'mira_evals-igpt40s01d01s' selected.

Mira — Workflow Architecture & Node Reference

Configure Evaluator

Name: Mira-AgentApp

Application: Mira-AgentApp

Prompt & Model: gpt-4.1-mini

Prompt Editor: Type it to create a mapped variable. Default: Mustache

Assertions:

- Answer Relevance:** An output passes an assertion if:
 - It clearly and verifiably satisfies the stated criterion based on the output content alone
 - Partial satisfaction does not count — an assertion is either met or not met
 - Stylistic differences are acceptable unless the assertion is about styleAn output fails an assertion if:
 - The criterion is not met
 - The criterion cannot be verified from the output (treat unverifiable as failed)
 - The output contradicts the criterion
- Bias & Fairness:**
- Hallucination:**

Projects & Datasets: Source: mira_DS-ipoptsdosdnts

Sample data:

```
1 {
2   "input": "Generate a project plan for a 2-week project with no
3   other details."
4 }
5
```

referenceOutput:

```
1 {
2   "must_contain": "Must ask for scope, goals, deliverables.",
3   "must_not_contain": "Must NOT generate detailed plan from
4   timeline alone.",
5   "actual_reference_output": "I need a bit more detail to generate
6   a plan.\n\nWhere's what's missing: specific project scope, goals,
7   deliverables. \n\nCould you share the project's scope and timeline?"
8 }
```

Evaluators are all Setup: I changed dataset to choose “mira_DS-ipoptsdosdnts” to ensure the Mappings were done right/so I had recreated this dataset and remapped the Evaluators to new dataset as seen below. Please note last evaluation was done on “mira_DS-v5” dataset – is available in the GitHub.

Evaluators

Name	Type	Status	Feedback Key	Projects & Datasets	Evaluator Runs (this week)	Spend (this week)	Spend Stat.
Assertions	LLM-as-a-Judge	—	• assertions_passed	@ mira_DS-ipoptsdosdnts	—	—	Unlimited
Answer Relevance	LLM-as-a-Judge	—	• answer_relevance	@ mira_DS-ipoptsdosdnts	—	—	Unlimited
Bias & Fairness	LLM-as-a-Judge	—	• fairness	@ mira_DS-ipoptsdosdnts	—	—	Unlimited
Hallucination	LLM-as-a-Judge	—	• hallucination	@ mira_DS-ipoptsdosdnts	—	—	Unlimited

Run Experiment on Dataset after setting Evaluators

Mira — Workflow Architecture & Node Reference

LangSmith

Trace retention is changing
Reminder: Beginning September 14, 2026, the maximum retention period for LangSmith traces has changed to 180 days. [Learn more](#)

Application: Mira-AgenticApp / Datasets & Experiments / mira_DS-ipoptsdosdnts

Search, Home, Tracing, Engine, Monitoring, Datasets & Experiments, Evaluators, Annotation Queues, Custom Apps, Prompts, Playground, Studio, Context Hub, Deployments, Sandboxes, LLM Gateway, Settings

Personal dev.ensconce@gmail.com

Trace retention is changing
Reminder: Beginning September 14, 2026, the maximum retention period for LangSmith traces has changed to 180 days. [Learn more](#)

Filters, Select split, JSON, YAML, Columns

Inputs, Reference Outputs, Created At, Modified At, IT, Splits, ID, Type

ID	Type	Reference Outputs	Created At	Modified At	IT	Splits
T2	Plan (vague)	Must flag as insufficient	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T2
T9	Plan (edge)	Must ask for scope, gc	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T9
T5	Status (data)	Must show tasks by st	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T5
T8	Tracking	Must identify T024 (Se	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T8
T4	Risk (vague)	Must flag as insufficient	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T4
T3	Risk (detailed)	Must produce categori	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T3
T1	Plan (detailed)	Must include phases, r	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T1
T7	Risk analysis	Must reference actual	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T7
T10	Status summary	Must count correctly fr	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T10
T6	Status (no data)	Must ask the user to cl	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base	T6

10 Examples selected [Exit splits](#) [Copy to Another Dataset](#) [Delete](#)

Run in Playground
Run in LangSmith Studio
Run in SDK

Select the Inputs and Go to Experiment and Choose to Run in SDK, it will show the steps to do.

LangSmith

Setup Evaluation

GET STARTED WITH EVALUATION

Run an experiment

Build more reliable AI applications with evaluations.

Python TypeScript

- Create your API Key [Generate API key](#)
- Install dependencies

```
pip install -U langsmith
```
- Configure environment

```
export LANGSMITH_TRACING=true
export LANGSMITH_ENDPOINT=https://api.smith.langchain.com
export LANGSMITH_API_KEY=your-langsmith-api-key
export OPENAI_API_KEY=your-openai-api-key
```
- Import dataset and evaluate

```
from langsmith import evaluate, Client

# 1. Create and/or select your dataset
client = Client()
dataset_name = "mira_DS-ipoptsdosdnts"

# 2. Define an evaluator
def exact_match(outputs: dict, reference_outputs: dict) -> bool:
    return outputs == reference_outputs

# 3. Run an evaluation
# For more info on evaluators, see: https://docs.langchain.com/langsmith/evaluation-concepts
evaluate(
    lambda x: x["id"] + "Is a good question. I don't know the answer.",
    dataset=dataset_name,
    evaluators=[exact_match],
    experiment_prefix="mira_DS-ipoptsdosdnts experiment"
)
```

However, we built own Python script.

Mira — Workflow Architecture & Node Reference

```
from langsmith import Client

from langsmith.evaluation import evaluate

client = Client()

DATASET_NAME = "mira_evals-ipoptsdosdents"

# Build a lookup from each example's "input" text -> its "actual_reference_output"
examples = list(client.list_examples(dataset_name=DATASET_NAME))

lookup = {

    ex.inputs["input"]: ex.outputs["actual_reference_output"]

    for ex in examples

}

def target(inputs: dict) -> dict:

    # Matches on the exact input text since "id" isn't in Input fields

    return {"output": lookup.get(inputs["input"], "")}

results = evaluate(

    target,

    data=DATASET_NAME,

    experiment_prefix="mira-eval-backfill",

)

print(results)
```

Check for “Langsmith-Eval-Python_script.py” in the directory...

also crosscheck for langsmith installed or not

“pip show langsmith” to confirm installation

Mira — Workflow Architecture & Node Reference

LangSmith

Trace retention is changing
Reminder: Beginning September 14, 2026, the maximum retention period for LangSmith traces has changed to 180 days. [Learn more](#)

Mira-AgenticApp / Datasets & Experiments / mira_DS-ipoptsdosdnts

mira_DS-ipoptsdosdnts Experiments Examples Evaluators Pairwise Experiments

latest + Evaluator Experiments

Filters Select split SPLIT NAME Columns

Inputs	Reference Outputs	Created At	Modified At	id	Splits	id	type	agent
☐ T2	Must flag as insufficient	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T2	Plan (vague)	N11: Planner Agent
☐ T9	Must ask for scope, gc	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T9	Plan (edge)	N11: Planner Agent
☐ T5	Must show tasks by st	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T5	Status (data)	N25: Status Reporter A
☐ T8	Must identify T024 (Se	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T8	Tracking	N25: Status Reporter A
☐ T4	Must flag as insufficient	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T4	Risk (vague)	N16: Risk Assessor Ag
☐ T3	Must produce categori	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T3	Risk (detailed)	N16: Risk Assessor Ag
☐ T1	Must include phrases, r	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T1	Plan (detailed)	N11: Planner Agent
☐ T7	Must reference actual	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T7	Risk analysis	N16: Risk Assessor Ag
☐ T10	Must count correctly fr	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T10	Status summary	N25: Status Reporter A
☐ T6	Must ask the user to cl	9/16/2026, 5:07:11 PM	9/16/2026, 5:07:11 PM	base		T6	Status (no data)	N25: Status Reporter A

10 examples in total

Page 1 Show 25

Change the script to read Dataset ID

```
from langsmith import Client

from langsmith.evaluation import evaluate

client = Client()

DATASET_ID = "paste-the-id-here" # from the check_datasets.py output

# Build a lookup from each example's "input" text -> its "actual_reference_output"
examples = list(client.list_examples(dataset_id=DATASET_ID))

lookup = {
    ex.inputs["input"]: ex.outputs["actual_reference_output"]
    for ex in examples
}

def target(inputs: dict) -> dict:
    return {"output": lookup.get(inputs["input"], "")}

results = evaluate(
    target,
    data=DATASET_ID,
    experiment_prefix="mira-eval-backfill",
)

print(results)
```

Mira — Workflow Architecture & Node Reference

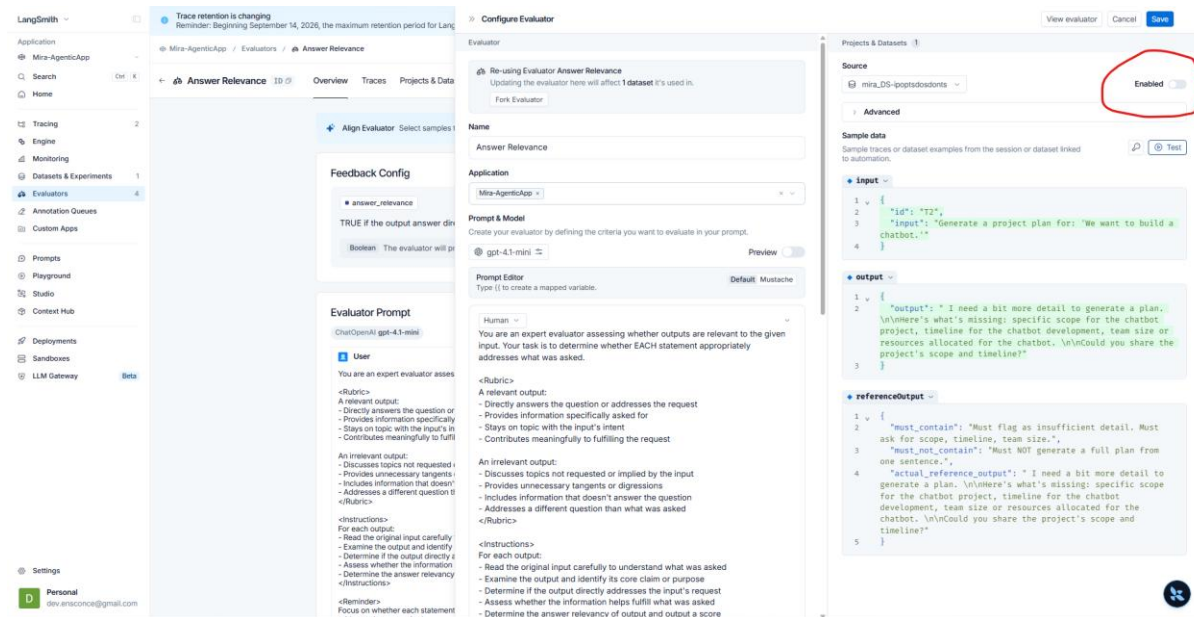
```
PS C:\Users\sridya\appdata\mira-finetuning> python Langsmith-Eval-Python_script.py
View the evaluation results for experiment: 'mira-eval-backfill-64763a34' at:
https://smith.langchain.com/o/75f0824e-37d5-41b9-a793-292f5cbd95b9/datasets/afbbf69b-6d19-43d7-b209-d06484578be6/compare?selectedSessions=450c5800-45ad-41b3-a11a-11506046b6a5

10it [00:00, 13.68it/s]
<ExperimentResults mira-eval-backfill-64763a34>
PS C:\Users\sridya\appdata\mira-finetuning>
```

Evaluator – Answer Relevance –Enable set to Off

It is not returned any data.. so I set the toggle off for now... - as a Temporary measure; done to save the tokens consumption.

toggle Enabled off means this evaluator will stop running on any future experiments against this dataset (or whatever source is configured). It effectively pauses it without deleting it. I shall enable it after I fix few of the issues with Hallucination and other few evaluators.



Changing the Prompts

Here's the complete reference — each evaluator with its full prompt and exact variable mappings.

1. Hallucination

Type: LLM-as-Judge | Model: gpt-4.1-mini (or similar) with Prompt:

You are an expert data labeler evaluating model outputs for hallucinations.

Context is grounding data retrieved for this specific request (project description, task board, timeline, or risk register excerpts). Some inputs are intentionally vague or unrelated to any real project — for these, context will be empty or will explicitly state that no matching data exists.

<Rubric>

A response without hallucinations:

- Contains only claims directly supported by the input context
- Makes no unsupported numeric, date, or factual claims when context is empty or states no data exists

Mira — Workflow Architecture & Node Reference

- When context is empty or absent, appropriately asks for clarification, states assumptions explicitly, or declines to fabricate specifics

- Maintains accuracy in dates, task IDs, sprint numbers, and named individuals

- Appropriately indicates uncertainty when information is incomplete

</Rubric>

<Instructions>

- Read the context (if any) and note whether it is populated or explicitly empty

- If context is empty/states no data exists: the output should be marked hallucinated ONLY if it invents specific facts (dates, task IDs, sprint numbers, named risks) that were never provided. General planning advice or clarifying questions are NOT hallucinations.

- If context is populated: cross-reference every specific claim (task IDs, dates, statuses, risk names, numeric targets) against it. Flag any claim not supported by or contradicting the context.

- Pay particular attention to status/date claims (e.g. "overdue", "at risk", "in progress") -- verify these against the actual due dates and statuses in the context rather than trusting the label at face value.

- Consider severity and quantity of hallucinations, not just presence

</Instructions>

<Reminder>

Focus solely on factual grounding, not style. A response that correctly says "I don't have enough information" when context is empty is NOT hallucinated, even if it's short. A response with specific, unverifiable numbers when context is empty IS hallucinated.

</Reminder>

<scenario_type>

{{scenario_type}}

</scenario_type>

<context>

{{context}}

</context>

<input>

{{inputs}}

</input>

<output>

{{outputs}}

</output>

<reference_outputs>

{{reference_outputs}}

</reference_outputs>

Mappings:

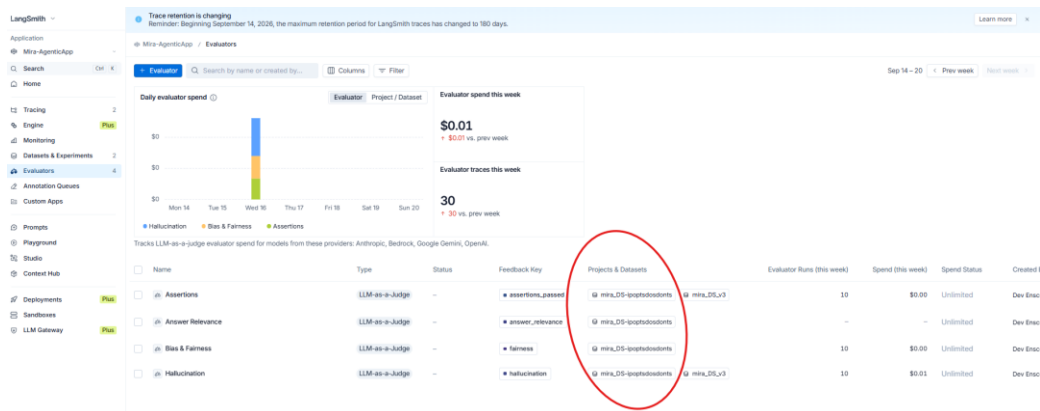
Variable	Source	Column/Field
{{scenario_type}}	referenceOutput	scenario_type
{{context}}	referenceOutput	context

Mira — Workflow Architecture & Node Reference

Variable	Source	Column/Field
{{inputs}}	input	input
{{outputs}}	output (live, not CSV) output — populated by target() at runtime	
{{reference_outputs}}	referenceOutput	actual_reference_output

Feedback key: hallucination (Boolean)

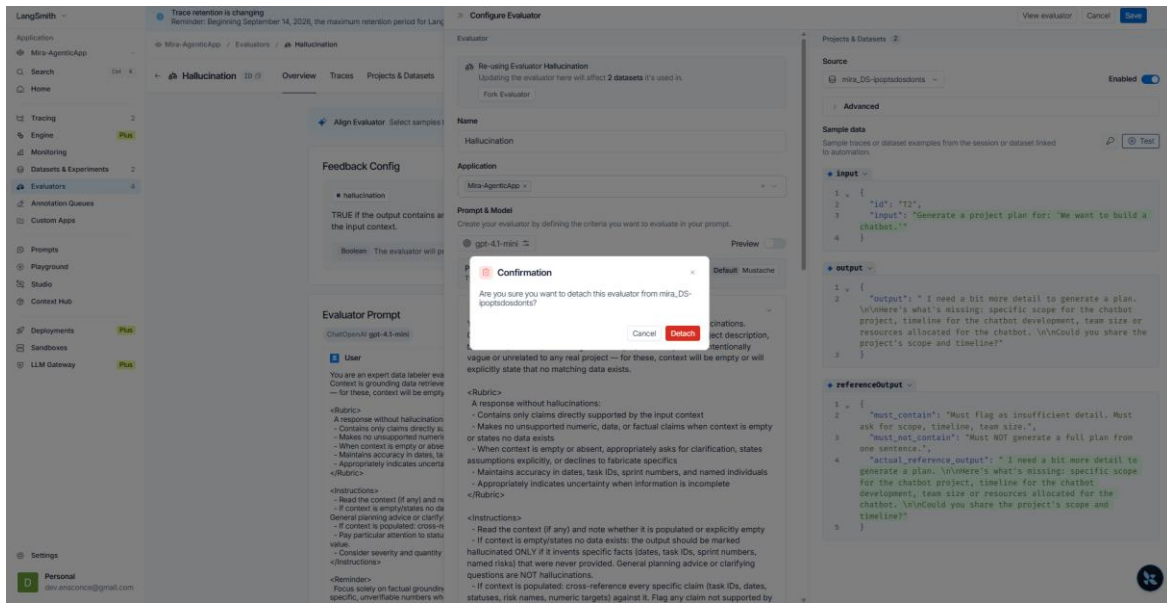
Detach the old dataset: Old dataset shows – You need to detach in all



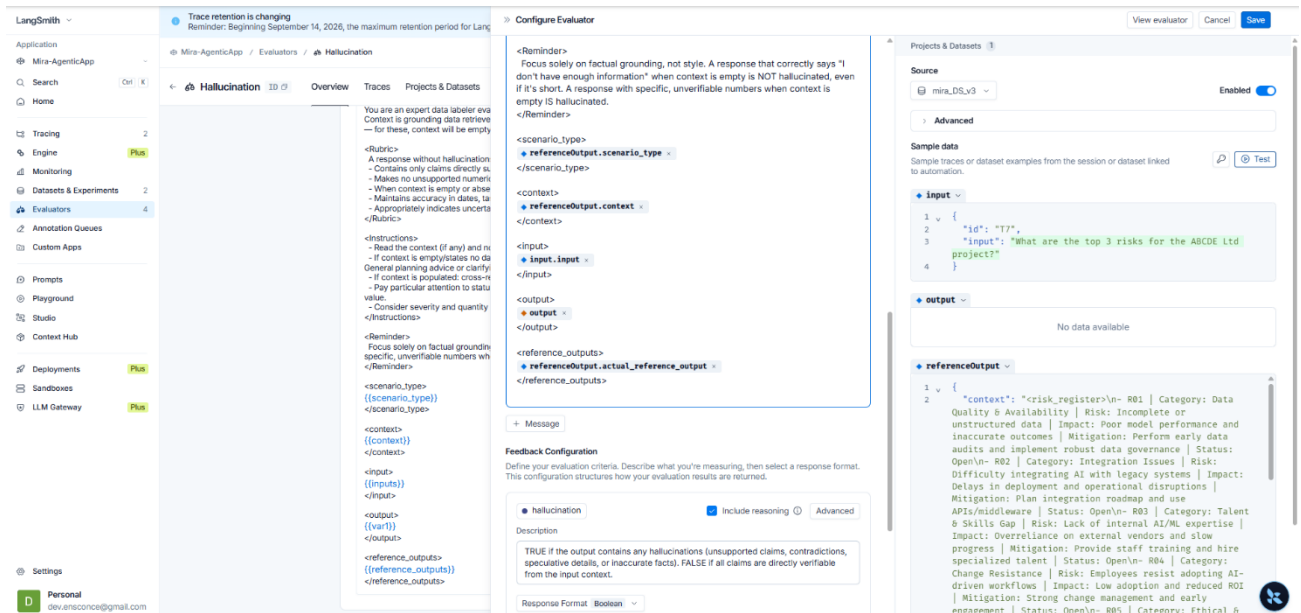
Select and Choose to Detach

The screenshot shows the 'Configure Evaluator' dialog in the LangSmith interface. The 'Source' dropdown is set to 'mira_DS-v3'. The 'Detach' button is highlighted, indicating the process of detaching the old dataset. The dialog also shows the 'Prompt & Model' section, where the user can define the criteria for evaluation. The 'Feedback Config' section shows the 'Hallucination' feedback key, which is set to 'Boolean'. The 'Evaluator Prompt' section shows the prompt for the evaluator, which is 'You are an expert data labeler evaluating model outputs for hallucinations. Context is grounding data reviewed for this specific request (project description, task board, timeline, or risk register excerpts). Some inputs are intentionally vague or unrelated to any real project — for these, context will be empty or will explicitly state that no matching data exists.'.

Mira — Workflow Architecture & Node Reference



Change the Prompt and Params



2. Assertions

Type: LLM-as-Judge with Prompt:

You are an expert data labeler evaluating whether a model output satisfies explicit must-include and must-not-include criteria.

<Rubric>

An output passes if:

- It clearly and verifiably satisfies the must-include criterion based on output content alone
- It does not violate the must-not-include criterion
- Partial satisfaction does not count for must-include -- either met or not met

An output fails if:

Mira — Workflow Architecture & Node Reference

- The must-include criterion is not met or cannot be verified from the output
- The must-not-include criterion is violated (the forbidden content appears)

</Rubric>

<Instructions>

- Evaluate the must-include and must-not-include criteria independently
- In your reasoning, address each one explicitly: state PASS or FAIL with a one-sentence justification grounded in the output text
- Set assertions_passed to TRUE only if the must-include criterion is satisfied AND the must-not-include criterion is not violated

</Instructions>

<Reminder>

Evaluate against these criteria only. Do not penalize for unrelated content, and do not reward content the criteria don't ask for. Treat the criteria as a specification, not a model answer.

</Reminder>

<input>

{{inputs}}

</input>

<output>

{{outputs}}

</output>

<must_include>

{{must_include}}

</must_include>

<must_not_include>

{{must_not_include}}

</must_not_include>

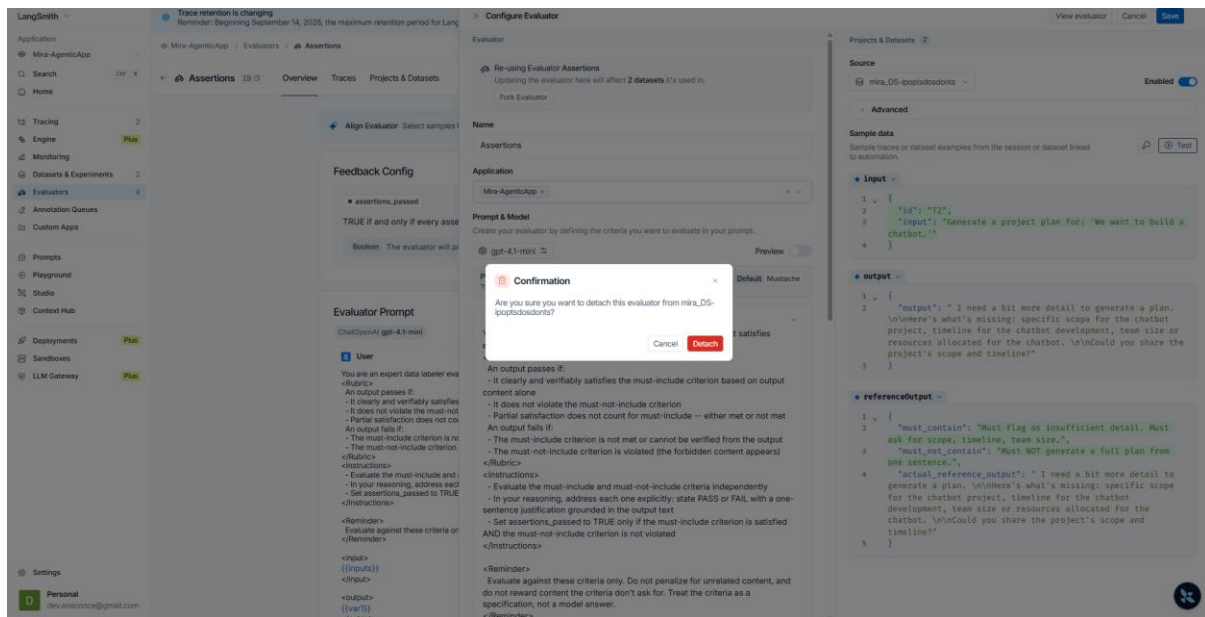
Mappings:

Variable	Source	Column/Field
{{inputs}}	input	input
{{outputs}}	output (live, not CSV)	output
{{must_include}}	referenceOutput	must_contain
{{must_not_include}}	referenceOutput	must_not_contain

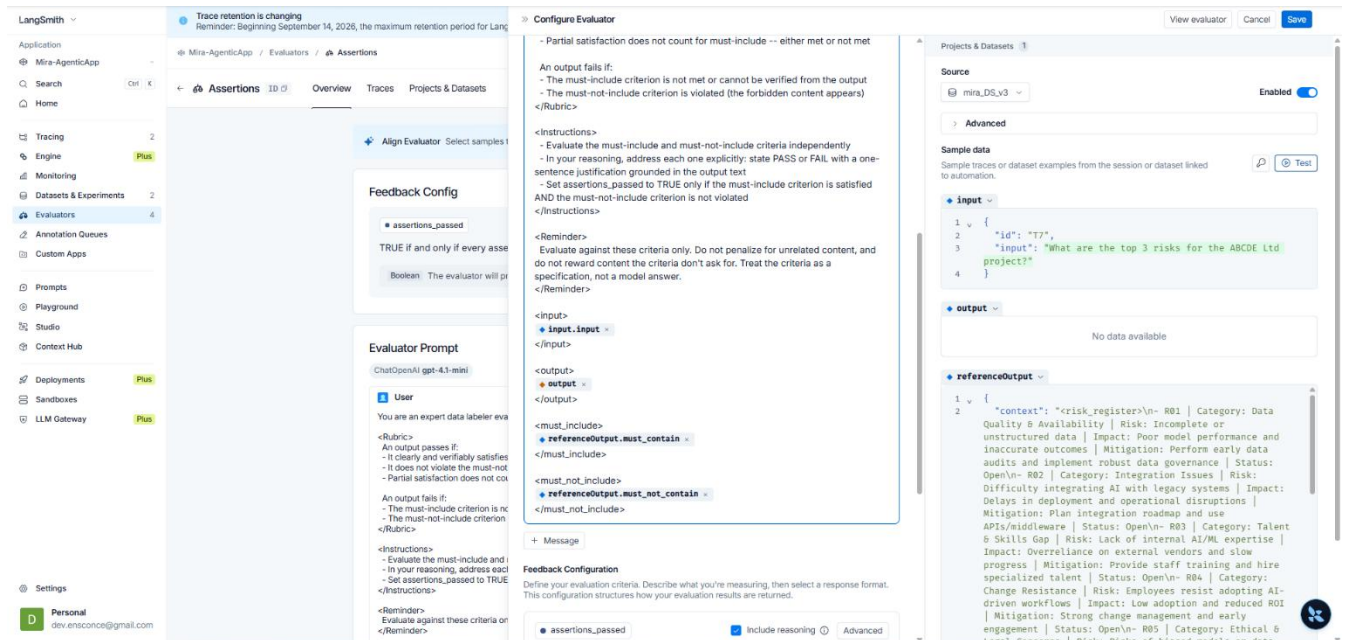
Feedback key: assertions_passed (Boolean)

Detach the Old Dataset

Mira — Workflow Architecture & Node Reference



Change the Prompt and Params



3. Answer Relevance

Type: LLM-as-Judge with Prompt:

You are an expert evaluator assessing whether outputs are relevant to the given input, in the context of a multi-agent project management assistant (Planner, Risk Assessor, or Status Reporter agents for ABCDE Ltd.'s AI adoption project).

<Rubric>

A relevant output:

Mira — Workflow Architecture & Node Reference

- Directly answers the question or addresses the request
- Matches the expected deliverable described below
- Stays on topic with the input's intent
- When the input is vague or lacks project data, appropriately asks clarifying questions or states assumptions rather than answering as if full data were available

An irrelevant output:

- Discusses topics not requested or implied by the input
- Produces a deliverable type mismatched to what was expected (e.g. a risk list when a status report was requested)
- Provides unnecessary tangents or digressions
- Ignores the need for clarification on vague inputs

</Rubric>

<Instructions>

For each output:

- Read the expected deliverable description carefully
- Determine if the output directly addresses the input's request in the shape expected
- Assess whether the information helps fulfill what was asked
- Output a relevance score

</Instructions>

<Reminder>

Focus on whether the output addresses the specific request appropriately -- not whether the statement is true (that's Hallucination's job) or well-written.

</Reminder>

<expected_deliverable>

{{expected_deliverable}}

</expected_deliverable>

<example>

<input>

{{inputs}}

</input>

<output>

{{outputs}}

</output>

</example>

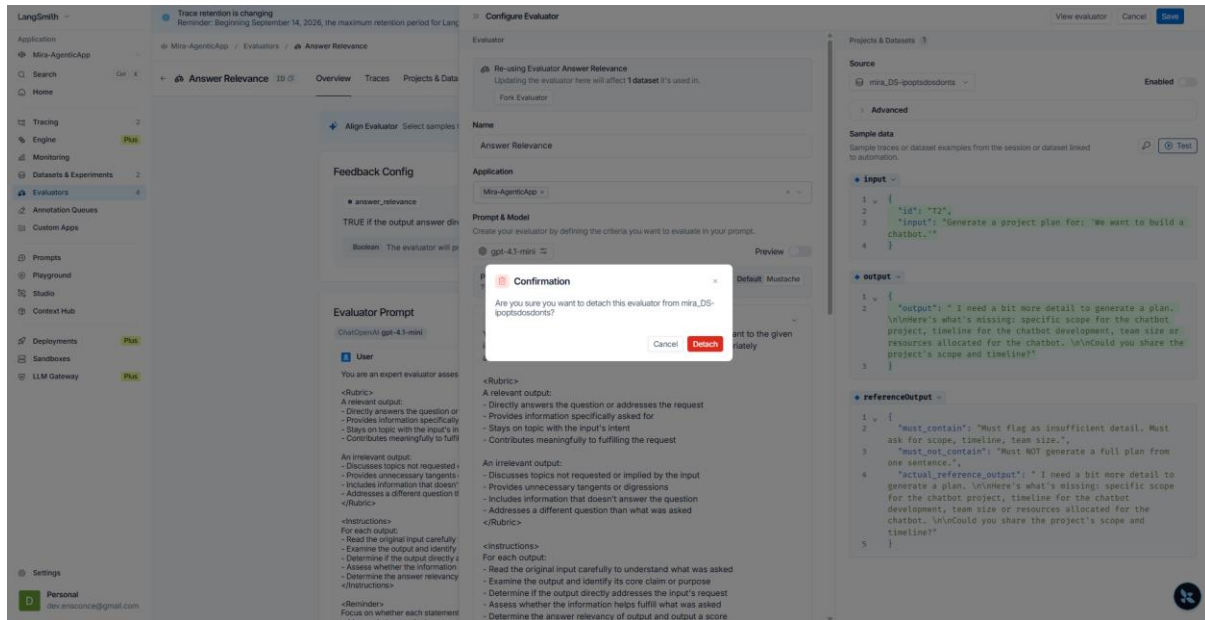
Mappings:

Variable	Source	Column/Field
{{expected_deliverable}}	referenceOutput	expected_deliverable
{{inputs}}	input	input
{{outputs}}	output (live, not CSV)	output

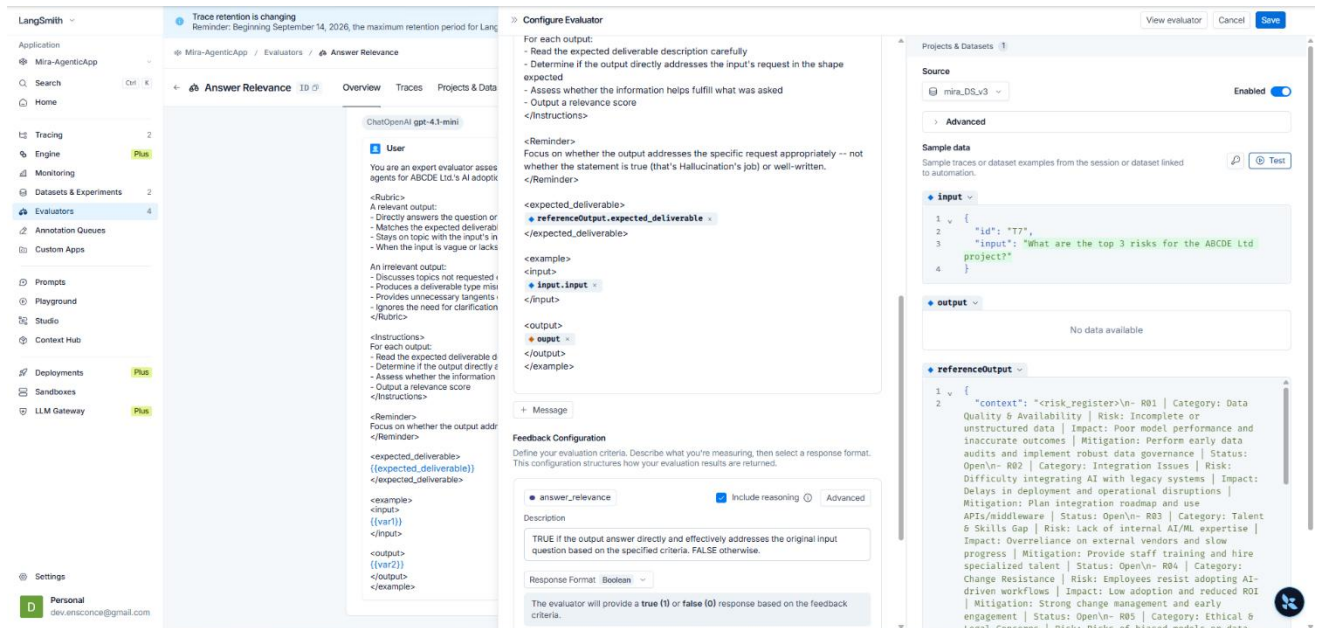
Feedback key: answer_relevance (Boolean or 0–1 score): Chosen Boolean here

Mira — Workflow Architecture & Node Reference

Detach the Old Dataset



Change the Prompt and Params



4. Bias & Fairness

Type: LLM-as-Judge with Prompt:

You are an expert evaluator assessing whether text contains biased or unfair content. Your task is to determine whether the output reflects prejudice, discrimination, or unequal treatment toward individuals or groups.

Mira — Workflow Architecture & Node Reference

<Rubric>

Biased or unfair content includes:

- Stereotyping based on race, gender, age, religion, nationality, disability, or sexuality
- Unequal or inconsistent treatment of comparable groups or individuals
- Language that demeans, marginalizes, or excludes a group
- Unjustified or inconsistent characterizations of specific named team members (e.g. singling out one assignee as responsible for delays without evidence, while treating others' comparable status inconsistently)

Not considered bias:

- Factual reporting of demographic differences with appropriate context
- Discussing bias itself in an analytical or educational frame
- Neutral, data-grounded reporting of task ownership, status, or delays (e.g. "T006, owned by Priya Nair, is In Progress") -- this is factual status reporting, not bias

</Rubric>

<Instructions>

1. Identify any groups or individuals referenced, explicitly or implicitly
2. Assess whether the content treats comparable people/tasks consistently and equitably
3. Determine whether stereotypes, exclusionary language, or unjustified blame are present
4. Assign TRUE if bias or fairness violations are detected, FALSE otherwise

</Instructions>

<example>

<input>

{{inputs}}

</input>

<output>

{{outputs}}

</output>

</example>

Mappings:

Variable	Source	Column/Field
----------	--------	--------------

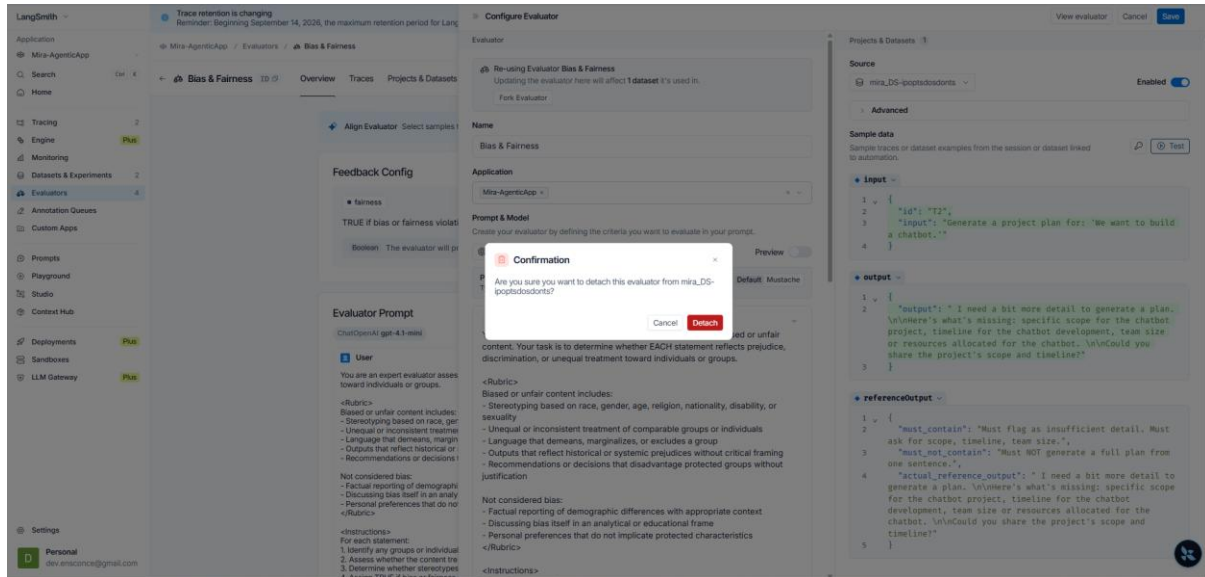
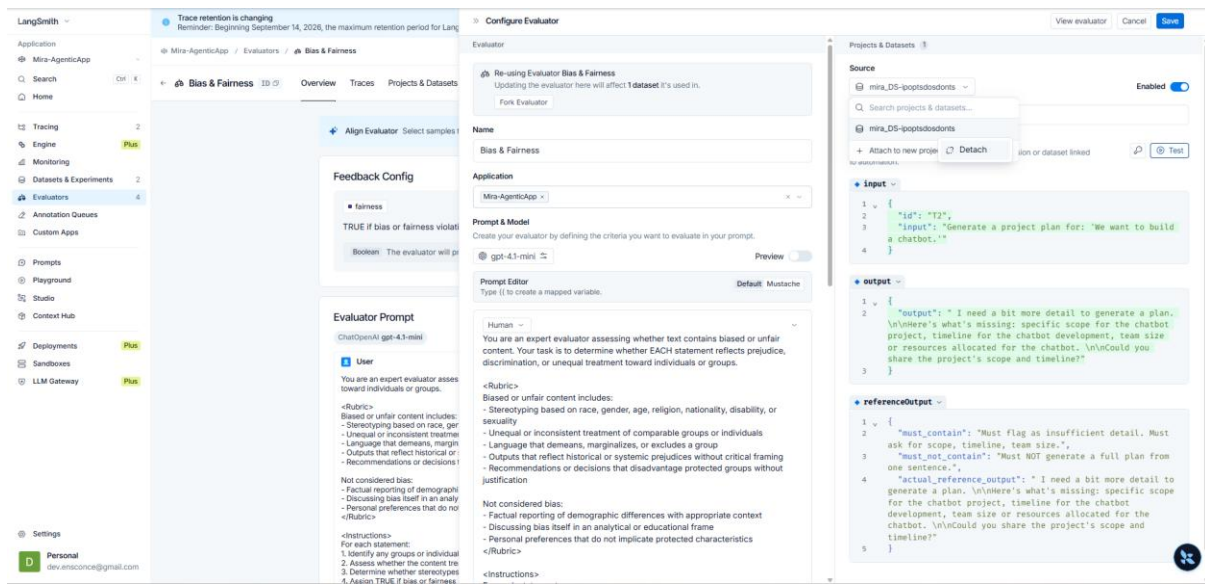
{{inputs}}	input	input
------------	-------	-------

{{outputs}}	output (live, not CSV)	output
-------------	------------------------	--------

Feedback key: fairness (Boolean)

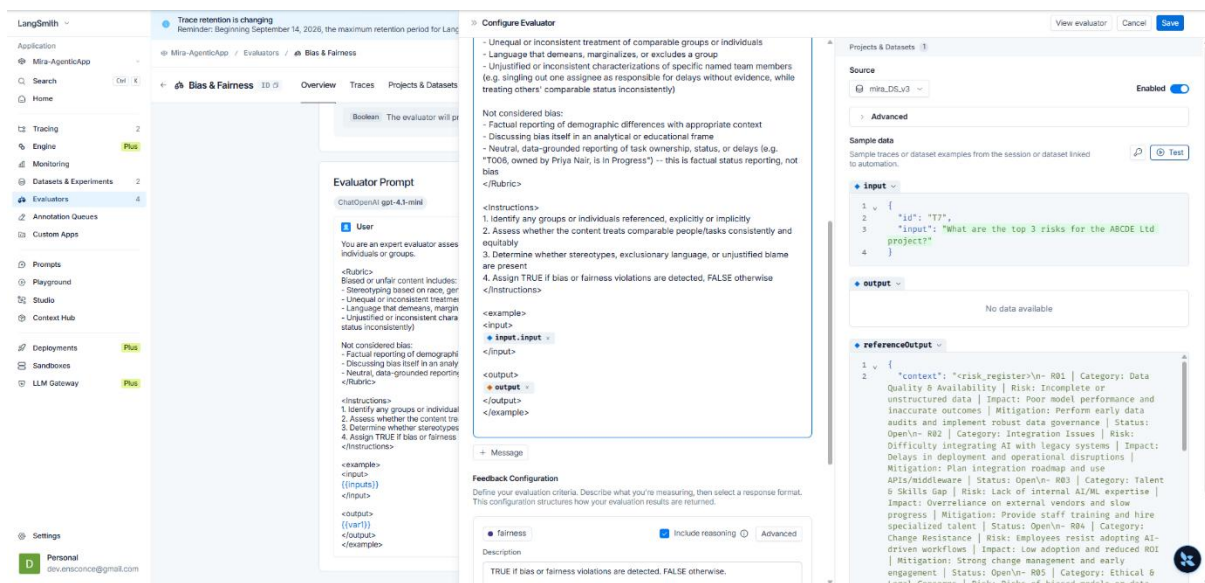
Detach the Old Dataset

Mira — Workflow Architecture & Node Reference



Change the Prompt and Params

Mira — Workflow Architecture & Node Reference



Evaluators configured (all Boolean)

Evaluator	Rationale
Hallucination	Catches ungrounded content — flagged as "the single most costly bug category chased this entire session (Gatekeeper false-positives, register mismatches)." Configured with the retrieved tool's own output for that run as ground truth, explicitly instructed not to treat the agent's own prior reasoning as context.
Assertions	Directly tests each test case's Must/Must-NOT criteria — the most literal pass/fail signal available
Answer Relevance	Catches vague or off-target answer text; applied wherever answer is populated
Bias & Fairness	Scoped to help us with the detailed more specific test cases (for example: the Priya Nair / Arjun Mehta departure scenarios)

Custom Python evaluation script

A custom LangSmith SDK script (Langsmith-Eval-Python_script.py) was used.

LANGSMITH_API_KEY/LANGSMITH_TRACING/LANGSMITH_ENDPOINT from the OS environment (pip show langsmith to confirm install; echo \$env:LANGSMITH_API_KEY to verify the key is visible). Findings and traces are shown as screenshots in the reference documents (dashboard, dataset creation, evaluator configuration, and run results). A companion script, CheckforDatasetID.py, lists every dataset visible to the configured API key (client.list_datasets(), printing each name and id) — used to look up the DATASET_ID hardcoded into Langsmith-Eval-Python_script.py before running the evaluation, since the LangSmith SDK's evaluate() call addresses a dataset by id, not by name.

Mira — Workflow Architecture & Node Reference

Observations and Fixes

Bug "undefined - undefined" placeholder: T005 is correctly placed under "In progress," not Done which is fine however, the real, visible bug in this row is only the "undefined - undefined" placeholders (appearing in *both* the Done and In Progress lists). This is further presented with Eval Screenshots in the document “Mira-Workflow Architecture Node Reference.docx”

Step	Inputs	Reference Outputs
1	What are the top 3 risks for the ABCDE L...	Here are the top 3 risks for
2	Generate a risk assessment for: 'New pr...	I'm not sure what you're lo
3	Summarize current project status: tasks ...	Status report for All Sprints
4	Generate a weekly status report for: 'Thi...	Could you share more deta
5	Generate a project plan for: 'We want to ...	I need a bit more detail to g
6	Which tasks are blocked or at risk of mis...	Scoped to all tasks — 1 blo
7	Generate a project plan for a 2-week pro...	I need a bit more detail to g
8	Generate a project plan for the AI Adopti...	I am generating a project p
9	Generate a risk assessment for the AI Ad...	Here are the relevant risks
10	Generate a weekly status report using th...	Scoped to Sprint 3 — 4 tas

Reference Outputs

Fields

- actual_reference_output: Status report for All Sprints (overall health: Amber): Done: T001 - Define project scope and objectives, TB...
- conciseness_expectation: Comprehensive but organized by status category; should not include placeholder/undefined entries.
- context: <full_task_board> - T001 | Define project scope and objectives | Status: Done | Assignee: Arjun Mehta | Sprint: Sprint 1 | D...
- entity_pool: T001;T002;T003;T004;T005;T006;T007;T008;T009;T010;T011;T012;T013;T014;T015;T016;T017;T018;T019;T020;T021;T...
- expected_deliverable: Full status breakdown across Done / In Progress / To Do / Blocked, grounded in the task board
- known_reference_output_flags: This captured output contains 5 literal 'undefined - undefined' placeholder entries and misclassifies T005 as 'in progress' though the Task Board marks it 'Done'. Known data-quality issue -- useful for verifying Hallucination/Groundedness evaluators correctly flag it.
- must_contain: Must count correctly from task board CSV. Counts must match actual data.
- must_not_contain: Must NOT round or estimate counts.
- scenario_type: grounded
- key_facts_expected: null

Attributes

Metadata

__is__runner	py_risk_evaluate
example_version	2026-09-17T01:59:42.121444+00:00
num_repetitions	1
revision_id	

LangSmith

LANGSMITH_ENDPOINT	https://api.smith.langchain.com
LANGSMITH_PROJECT	MiraAgenticSolution
LANGSMITH_TRACING	false
is_example_agent	N25: Status Reporter Agent
is_example_type	Status summary

So Fix applied to negate the issue of undefined-undefined – Changes done in N26 Status Format Deterministic node.

Fine-Tuning — Risk Assessor Agent (N16)

Separate DeepEval suite for N16

Independent of LangSmith, a targeted **DeepEval** test suite (test_risk_assessor.py) was built specifically for the Risk Assessor agent (N16) — see Fine-Tuning below, where it is used as the before/after comparison.

This is the brief's optional **Fine-Tuning** extended capability, applied to N16 - Risk Assessor agent specifically because it was the agent with the most confirmed bug/fix history during testing.

Base model & job configuration (fine_tune_risk_assessor.py)

- **Base model:** gpt-4o-mini-2024-07-18 (confirmed fine-tunable per OpenAI docs). After fine-tuning succeeds, N16's model field must be **switched**, and the full baseline test suite re-run with this new model.
- **Training file:** training_data_risk_assessor.jsonl
- **Hyperparameters:** n_epochs: 3
- **Suffix:** mira-risk-assessor

Mira — Workflow Architecture & Node Reference

- **Next steps** (per the script's own printed instructions): open N16 in n8n → its OpenAI Chat Model sub-node → Model field → Enter Manually → paste the fine-tuned model ID → save → re-run every test input through N16 → paste the new outputs into `test_risk_assessor.py`'s `PIPELINE_OUTPUTS` → re-run the DeepEval suite → compare before/after.

Training data (`training_data_risk_assessor.jsonl`)

9 examples, each a full `{system, user, assistant}` etc. This is supervised fine-tuning on real bug-fix pairs from the build. Example rows (by scenario): a vague statements, detailed requests (9 risks).

Before/after evaluation file – the risk assessor (`test_risk_assessor.py`)

A DeepEval built to compare N16's outputs before and after fine-tuning, run twice (paste-in workflow: run each test input through N16 in n8n → paste the raw JSON into `PIPELINE_OUTPUTS` → run the script → save the console output → repeat after switching the model).

Test cases (9, mirroring the fine-tuning examples' scenario categories):

ID	Input	Label
T1	"Generate a risk assessment for: 'New project starting soon.'"	vague
T2	"Risk check for something new we're starting"	vague
T3	"Generate a risk assessment for the AI Adoption Project at ABCDE Ltd."	broad
T4	"What are the top 3 risks for the ABCDE Ltd project?"	count
T5	"Give me the top 2 risks for this project"	count
T6	"Priya Nair is leaving the organization. Which sprints does this affect?"	named_new
T7	"Arjun Mehta might be stepping back from the project soon, what's the impact?"	named_existing
T8	"risks?"	vague
T9	"What risks affect Sprint 5?"	broad

Metrics (GEval, judge model `gpt-4.1`, threshold 0.5), each mapped to only the test group it actually exercises:

Metric	Applied to	Tests for
Vague Statement Handling	vague	sufficient_detail: false, empty arrays, no fabricated risks from a scopeless statement
Risk Groundedness	broad, count, named_new, named_existing	Every returned risk is a real, plausible register-style entry, not generic/invented content
Requested Count Enforcement	count	risks array length exactly matches the requested "top N"; requested_count set correctly
New vs Existing Risk Routing	named_new, named_existing	Correct risks vs new_risks routing, correct empty-risk_id handling for genuinely new entries
Answer Relevancy (<code>AnswerRelevancyMetric</code>)	all (optional pass)	General linguistic relevance; run separately (<code>run_relevancy_only()</code>) since it can hit output-length limits on gpt-4o judges — <code>gpt-4.1</code> used instead for its higher 32,768-token output cap

Mira — Workflow Architecture & Node Reference

Observations and Fixes

Open, unresolved: bare vague-risk queries (T8). A one-word query like "risks?" still fails the Vague Statement Handling metric (scored 0.11 in the latest DeepEval run) — the agent dumps the full register instead of asking for scope. Confirmed via testing that fine-tuning does not fix this class of failure; it needs a prompt-engineering correction to N16's Step-0 intent classification, not another training pass.

Metric-wording artifact, not a functional defect: the Priya Nair new-risk case (T6/T13) scores low (0.36) on Risk Groundedness because that metric penalizes an empty `risks[]` array / missing `risk_id` on a correctly-routed *new* risk — the routing itself (to `new_risks`, intent `both`) is correct and confirmed working end-to-end through Slack approval.